

UNITED STATES PATENT APPLICATION

System and Method for Automated Frequency Coordination-Integrated Radio Resource Management in 6 GHz Wireless Access Networks

AFC-Integrated RRM Patent — Technology Disclosure

Application Number: [PENDING]

Filing Date: July 2026

Assignee: Hewlett Packard Enterprise (HPE)

Technology Area: 6 GHz Wi-Fi 6E / Wi-Fi 7, Cloud RRM, Automated Frequency Coordination

Repository: AFC_Patent_Final — Cloud RRM Platform

Name / Team	Organization	Department
Mist Systems Engineering Team	HPE (Hewlett Packard Enterprise)	Wireless Engineering

Table of Contents

1. Cover Page
2. Table of Contents
3. Abstract
4. Field of the Invention
5. Background of the Invention
6. Uniqueness Compared to Prior Art
7. Prior Art Comparison Table
8. Summary of the Invention (10 Novel Contributions)
9. Detailed Description — Part A: AFC Core Architecture
 - Figure 1 — Overall System Architecture
 - Figures 2–4 — AFC Lifecycle, Payload, Response
 - Figures 5–8 — Channel Map, EIRP
 - Figures 17–19 — AP Lifecycle, Redis, Multi-Provider
10. Detailed Description — Part B: RRM-AFC Integration
 - Figures 12–13 — Storm Bolt, State Machine
 - Figures 20–22 — LPI, ACS+AFC, afcNode
11. Detailed Description — Part C: Geolocation, EIRP, Advanced Channel Logic
 - Figures 9–11 — Geolocation
 - Figures 14–16 — 320MHz, ch_delta, DeleteAP
12. Implementation Reference Table
13. Claims
14. Abstract of the Disclosure

Abstract

*A system and method are disclosed for integrating Automated Frequency Coordination (AFC) with cloud-scale Radio Resource Management (RRM) in 6 GHz wireless access networks. An AFC proxy intermediary service receives device registration payloads from a cloud RRM engine, forwards spectrum availability inquiries to regulatory AFC systems, and returns per-channel maximum Effective Isotropic Radiated Power (EIRP) assignments to access points (APs). The system derives channel-specific EIRP limits by computing the minimum of the AFC-reported per-channel maximum EIRP and a Power Spectral Density (PSD)-derived EIRP value, expressed as $\text{maxPsd_dBm_per_MHz} + 10 * \log_{10}(\text{channel_bandwidth_MHz})$, subject to a minimum PSD floor indexed by channel bandwidth. Geolocation validation employs WGS84-to-ECEF coordinate transformation with a configurable displacement threshold to detect AP relocation events, triggering automatic AFC re-registration. The system handles abnormal AP lifecycle events including unclaimed and unassigned states by purging device registrations from the AFC proxy and associated geolocation caches. A fallback mechanism degrades AFC-capable APs to Low Power Indoor (LPI) operation when GPS coordinates are unavailable, preserving connectivity. Integration with a distributed stream-processing topology enables real-time, per-AP AFC coordination across enterprise-scale wireless deployments.*

Field of the Invention

The present invention relates to wireless local area network (WLAN) management systems, and more particularly to systems and methods for automatically coordinating spectrum access in the 6 GHz frequency band (5925 MHz to 7125 MHz) through integration of Automated Frequency Coordination (AFC) mechanisms with cloud-managed Radio Resource Management (RRM) platforms operating under regulatory frameworks including United States Federal Communications Commission (FCC) rules codified at 47 CFR Part 15, Subpart E.

The invention further relates to:

- Standard Power (SP) access point operation at elevated transmit power levels in the 6 GHz band, as distinguished from Low Power Indoor (LPI) operation;
- Dynamic per-channel EIRP limit enforcement derived from real-time AFC system responses;
- Geolocation-aware spectrum coordination utilizing WGS84 Earth-Centered, Earth-Fixed (ECEF) coordinate transformation;
- Cloud-scale distributed stream processing for enterprise wireless network management;
- Automated device lifecycle management within AFC registration systems.

Background of the Invention

1. The 6 GHz Band and Incumbent Protection Problem

The Federal Communications Commission (FCC) allocated 1,200 MHz of spectrum in the 6 GHz band (5925-7125 MHz) for unlicensed Wi-Fi use under 47 CFR Part 15, Subpart E, effective April 2020. This allocation enables high-throughput IEEE 802.11ax (Wi-Fi 6E) and IEEE 802.11be (Wi-Fi 7) networks supporting channel bandwidths of 20, 40, 80, 160, and 320 MHz.

However, this spectrum is also occupied by incumbent licensed services whose operations must be protected from interference. These incumbents include:

- **Fixed Satellite Service (FSS)** earth stations receiving transmissions from geostationary satellites in C-band and Ku-band;
- **Fixed Microwave Links** used by utilities, public safety, and backhaul operators;
- **Broadcast Auxiliary Service (BAS)** and **Cable Television Relay Service (CARS)** links;
- **Earth Exploration Satellite Service (EESS)** receivers.

Unlicensed 6 GHz devices operating at elevated power levels without coordination risk causing harmful interference to these incumbents, rendering affected satellite ground stations and microwave links inoperable. The geographic distribution of incumbents is non-uniform and dynamic, requiring real-time database-driven coordination rather than static exclusion zones.

2. Regulatory Framework: 47 CFR Part 15, Subpart E

The FCC established a tiered regulatory framework for 6 GHz unlicensed devices:

Tier 1 — Low Power Indoor (LPI) Operation:

- Maximum EIRP: 30 dBm (1 watt) for standard client devices
- Maximum power spectral density: 5 dBm/MHz
- Permitted indoors only; no AFC required
- Cannot be used by devices with external antennas in most configurations

Tier 2 — Standard Power (SP) Operation:

- Maximum EIRP: 36 dBm (approximately 4 watts)
- Maximum power spectral density: 23 dBm/MHz
- **Requires AFC coordination** before transmitting
- Permitted indoors and outdoors
- Enables significantly greater coverage range and throughput compared to LPI

Tier 3 — Very Low Power (VLP) Operation:

- Maximum EIRP: 14 dBm
- No AFC required; limited to portable devices

The critical constraint of SP operation — and the central regulatory mandate addressed by the present invention — is that a Standard Power device **must not transmit** until it has received authorization from a registered AFC system specifying which channels are available and at what maximum power levels at the device's specific geographic location.

3. AFC System Architecture

The FCC requires that AFC systems be independently certified and administered. The Wi-Fi Alliance (WFA) manages a testing and certification process for AFC systems. The AFC system architecture, as standardized, involves:

1. **The AFC System** (also called the AFC Database): A certified backend service that maintains records of incumbent locations, antenna patterns, propagation models, and exclusion zones. The AFC system receives spectrum inquiries and returns channel availability responses.
1. **The Standard Power Device**: An access point or client device that submits its geolocation, device identity, and spectrum requirements to the AFC system before operating.
1. **The AFC Protocol**: Based on the OpenAFC API specification, requests include device descriptor (serial number, FCC certification ID, ruleset), geographic location (ellipse with center latitude/longitude, semi-major axis, semi-minor axis, orientation, and elevation above ground level), and inquired channels or frequency ranges.
1. **The AFC Response**: Returns a list of available channels per operating class with per-channel maximum EIRP values, frequency range availability, and an expiration timestamp after which re-inquiry is required.

4. Limitations of Prior Art

Prior art systems for 6 GHz AFC coordination suffer from several significant limitations:

4.1 Static Channel Configuration

Prior art wireless management systems that predate AFC integration rely on static channel lists configured by network administrators. These systems cannot dynamically adapt channel and power assignments to real-time AFC responses, resulting in either overly conservative power settings that reduce coverage or potential regulatory violations when incumbent databases are updated.

4.2 Per-Device AFC Without RRM Coordination

Early 6 GHz AP firmware implementations perform AFC queries independently, without coordination from a centralized RRM engine. This leads to inconsistent channel selections across APs within the same site, creating co-

channel interference and suboptimal spectrum utilization. A centralized RRM system that receives and incorporates AFC responses can optimize channel assignments across the entire site topology.

4.3 Absence of Geolocation Change Detection

Prior art AFC implementations submit geolocation once during initial configuration and do not detect or respond to AP physical relocation. When an AP is moved to a new location — even one with a different incumbent profile — it continues operating on the previously authorized channels until the AFC authorization expires. This creates a gap period during which the device may cause interference to incumbents near the new location that were not present near the original location.

4.4 No Graceful Degradation on GPS Failure

Prior systems that require GPS for AFC queries simply disable the 6 GHz radio when GPS coordinates are unavailable. This approach results in complete service outage for users dependent on 6 GHz connectivity, rather than a graceful fallback to lower-power LPI operation that does not require AFC but still provides connectivity.

4.5 Monolithic AFC-RRM Integration

Prior systems couple AFC operations directly within the AP firmware or within a monolithic network management platform. This architecture cannot scale to enterprise deployments spanning thousands of APs across hundreds of sites, where AFC queries, responses, and RRM decisions must be processed in parallel with sub-second latency.

4.6 Lack of Lifecycle Management

When APs are decommissioned, reassigned, or unclaimed, prior art systems do not purge the associated AFC registrations from the AFC proxy or AFC system databases. This results in stale device records that consume database resources and may interfere with future registrations for the same MAC address at a different location.

4.7 Coarse EIRP Limit Application

Prior art implementations apply the AFC-reported per-channel maximum EIRP directly as the transmit power limit, without accounting for the more restrictive PSD-based limits that may apply to the same channel. The FCC rules require that BOTH the per-channel EIRP limit AND the PSD limit be satisfied. Failure to compute the minimum of these two constraints may result in regulatory violations on channels where the PSD limit is more restrictive than the channel EIRP limit.

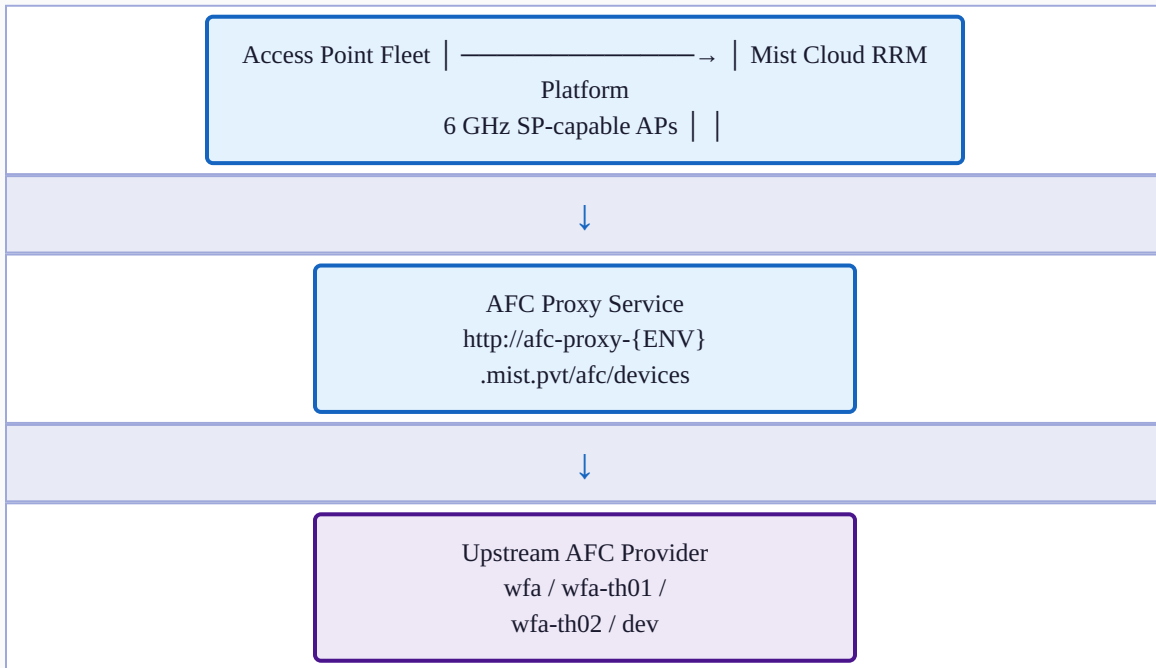
5. Summary of Problems Addressed

The present invention addresses the following specific technical problems not solved by prior art:

1. How to dynamically integrate real-time AFC channel and EIRP authorizations into a cloud-scale, multi-site RRM engine that simultaneously manages thousands of access points;
1. How to detect AP geolocation changes and automatically trigger AFC re-registration without requiring manual administrative action;
1. How to compute the correct maximum transmit power for each AFC-authorized channel by taking the minimum of the AFC-reported EIRP limit and the PSD-derived EIRP limit, enforced above a minimum floor value per bandwidth;
1. How to manage the complete lifecycle of AP registrations in the AFC system, including automatic deletion of registrations for APs that become unclaimed or unassigned;
1. How to provide graceful service degradation when GPS coordinates are unavailable, falling back to LPI operation rather than disabling the radio entirely;
1. How to handle 320 MHz channel assignments where the same 20 MHz sub-channel may belong to either of two non-overlapping 320 MHz channel plans (Mode 1: centers at channels 31, 95, 159; Mode 2: centers at channels 63, 127, 191) and resolve ambiguity using proximity-based mode classification;
1. How to support multiple AFC provider backends (production, test harness variants, and development environments) through a configurable provider flag architecture without modifying device firmware.

Figure 1 — Overall System Architecture

Before: This diagram shows the top-level actors that participate in cloud-managed AFC authorization. The actors include the AP fleet, Redis data stores, the Storm-based RRM control plane, the AFC proxy, and the upstream AFC provider, and the arrows show how authorization state and channel decisions move among them.



After: The figure highlights that the inventive locus is the cloud RRM platform rather than AP firmware. That architectural shift enables site-policy routing, fleet-scale state tracking, and controller-driven enforcement that tie directly into the detailed workflows described in the following figures.

Uniqueness Compared to Prior Art

The prior-art material extracted from `AFC_Patent_full_info.pdf` shows that Qualcomm, Federated Wireless, CommScope, Intel, Arris, Cisco, and MediaTek families generally emphasize one of three themes: device-side AFC request/response behavior, AFC-server incumbent-protection calculations, or special-purpose mobility and negotiated-grant workflows. Those references are important background, but they do not disclose the particular cloud-RRM control architecture implemented here.

First, the present invention moves AFC from a device-local or chipset-local routine into a cloud-scale Radio Resource Management control plane. The code-derived architecture described throughout the repository places AFC orchestration in `src/rrmAFC.py`, `src/rrmACSV2.py`, and `src/rrmHandler.py`, with the AFC proxy communication isolated in `src/afc/afc_query.py`. Prior art such as Qualcomm's foundational AFC request-response family and Intel's AFC allocation scheme focuses on how a device obtains authorization and behaves after authorization; it does not teach the specific use of Apache Storm bolts, Redis-backed state, and per-site policy routing to coordinate large numbers of APs without firmware changes.

Second, the invention introduces a geolocation validation mechanism that is more than simple location awareness. In `src/afc/afc_utils.py`, the `lla_to_ecef()` and `location_match()` flow transforms latitude and longitude into WGS84 Earth-Centered, Earth-Fixed coordinates and compares old and new registrations against a 200-meter site-policy threshold. The prior-art survey includes location-aware systems and iterative uncertainty tuning, but it does not disclose this ECEF-based cache-invalidation trigger embedded inside a cloud RRM re-registration workflow.

Third, the present implementation enforces a dual-constraint power calculation that combines AFC maxEirp with PSD-derived EIRP and then applies a bandwidth-specific minimum floor. The implementation reference in `src/afc/afc_utils.py` performs `EIRP_from_PSD = maxPsd_dBm_per_MHz + 10 * log10(channel_bandwidth_MHz)` (base-10 logarithm) and then enforces `ch_max_power = max(AFC_MIN_PSD[bandwidth], int(min(psdEirp[ch], maxEirp[j])))`. The prior-art references discuss channel or power assignment generally, but they do not teach this exact minimum-of-two-constraints plus floor-enforcement pipeline in a cloud-managed AFC controller.

Fourth, the repository discloses operational fallback and lifecycle behaviors absent from the prior-art survey. The `lpi_ok` path in `src/rrmACSV2.py` preserves service by downgrading an AP to Low Power Indoor operation when GPS is unavailable. The AP abnormal-state handler in `src/afc/afc_utils.py` and the DELETE retry logic in `src/afc/afc_query.py` automatically remove stale registrations for AP_UNCLAIMED and AP_UNASSIGNED devices. Prior art typically assumes either valid location data or device-side shutdown; it does not describe this cloud-managed fallback and cleanup sequence.

Fifth, the invention solves controller-level channel-plan issues that are not addressed in the surveyed AFC patents. The code-derived design includes 320 MHz Mode 1 and Mode 2 disambiguation, sub-channel expansion by `ch_delta`, and provider-flag routing among `wfa`, `wfa-th01`, `wfa-th02`, and `dev` without modifying AP

firmware. OpenAFC-style server implementations focus on returning authorizations, not on how a cloud RRM engine disambiguates overlapping 320 MHz layouts or swaps providers by policy flag across a fleet.

Prior Art Comparison Table

Feature	Prior Art	This Invention
-----	-----	-----
AFC control location	Device firmware, chipset behavior, or AFC-server logic dominates the surveyed references.	Cloud RRM coordinates AFC through <code>rrmAFC.py</code> , <code>rrmACSV2.py</code> , and the AFC proxy interface.
Geolocation change handling	One-time location awareness or iterative uncertainty tuning is discussed, but not cloud-side ECEF cache invalidation.	<code>lla_to_ecef()</code> and <code>location_match()</code> compare current and registered locations and trigger re-registration when displacement exceeds 200 meters.
Power enforcement	Prior art generally applies returned power limits or discusses channel/power behavior at a high level.	The implementation computes <code>EIRP_from_PSD = maxPsd_dBm_per_MHz + 10 * log10(channel_bandwidth_MHz)</code> and then applies <code>ch_max_power = max(AFC_MIN_PSD[bandwidth], int(min(psdEirp[ch], maxEirp[j])))</code> .
PSD floor enforcement	No surveyed reference states a bandwidth-indexed enforcement table integrated into controller-side RRM logic.	<code>AFC_MIN_PSD = {20: 14, 40: 17, 80: 20, 160: 23, 320: 26}</code> is enforced per bandwidth.
GPS failure response	Device disablement or failure handling is assumed, with no explicit LPI downgrade in the surveyed materials.	<code>lpi_ok</code> enables an LPI fallback path that preserves service and auditability.
AP lifecycle cleanup	Stale registration cleanup is not described as a controller-managed AFC lifecycle.	AP abnormal events automatically invoke <code>DeleteAP</code> and purge <code>devices/{apID}/gps</code> .
320 MHz layout logic	The surveyed references do not disclose controller-side Mode 1 / Mode 2 disambiguation for overlapping 320 MHz sub-channels.	The implementation uses Mode 1 centers <code>[31, 95, 159]</code> , Mode 2 centers <code>[63, 127, 191]</code> , and mode-aware sub-channel propagation.
Provider switching	Prior art assumes a fixed AFC provider or server endpoint.	Policy flags route requests among <code>wfa</code> , <code>wfa-th01</code> , <code>wfa-th02</code> , and <code>dev</code> without AP firmware change.

Feature	Prior Art	This Invention
Cloud-scale processing	Prior art may mention controllers or APs, but not an Apache Storm bolt lifecycle around AFC.	<code>RrmAFCBolt</code> executes event-driven and tick-driven AFC orchestration across a fleet.
State persistence	Most references focus on grants, channels, or incumbent calculations, not cloud cache design.	Redis stores GPS data, AFC status, expiry, and retry queues for long-lived lifecycle management.

Summary of the Invention

(10 Novel Contributions)

The present invention provides a system and method for integrating Automated Frequency Coordination (AFC) with cloud-managed Radio Resource Management (RRM) in 6 GHz wireless access networks. The invention introduces several novel technical contributions, each described below.

Novel Contribution 1: AFC Proxy Intermediary Architecture

The invention introduces an AFC proxy service that sits between the RRM cloud engine and the AFC system. Rather than having each access point independently query the AFC system — which would require each AP to maintain AFC credentials, handle retries, and manage response parsing — the AFC proxy centralizes all AFC interactions. The RRM engine submits device registration requests to the AFC proxy using a standardized JSON payload format. The AFC proxy maintains per-device state, forwards requests to the upstream AFC system, and returns processed channel availability information to the RRM engine.

The AFC proxy is addressed via a configurable URL pattern of the form:

...

`http://afc-proxy-{ENV}.mist.pvt/afc/devices`

...

where `{ENV}` is the deployment environment identifier (e.g., `staging`, `prod`, `eu`). This environment-parameterized addressing enables seamless promotion of RRM software across deployment tiers without reconfiguration of AFC endpoints.

Novel Contribution 2: Structured AFC Payload Construction

The invention provides a structured payload construction method that assembles a standards-compliant AFC request from AP configuration data. The payload includes:

- **Request identification:** `requestId` set to the AP's MAC address (enabling direct correlation between request and response)
- **Site identification:** `siteId` set to the network site UUID
- **Provider routing:** `providerName` resolved from a configurable flag ("0"->"wfa", "1"->"wfa-th01", "2"->"wfa-th02", "dev"->"dev")
- **Device descriptor:** `serialNumber` (AP MAC address) and `certificationId` array containing the FCC ID (e.g., "2AHBN-AP64") with ruleset identifier "US_47_CFR_PART_15_SUBPART_E"

- **Location:** WGS84 ellipse with center latitude/longitude, semi-major axis (capped at 325 meters), semi-minor axis (capped at 325 meters), orientation (normalized to 0°-180°), height above ground level (minimum 0.1 meters), vertical uncertainty, and indoor deployment indicator

- **Inquired frequency range:** 5925-7115 MHz (full 6 GHz SP band)

- **Inquired channels:** Global Operating Classes 131, 132, 133, 134, and 137

Novel Contribution 3: Dual-Constraint EIRP Computation

The invention introduces a dual-constraint EIRP computation method that derives the correct maximum transmit power for each AFC-authorized channel. For each available channel reported in the AFC response, the system:

1. Retrieves the AFC-reported per-channel maximum EIRP value from `availableChannelInfo[].maxEirp[]`
2. Computes the PSD-derived EIRP from the available frequency information using:

EIRP_from_PSD = maxPsd_dBm_per_MHz + 10 * log10(channel_bandwidth_MHz), where `log10` denotes the base-10 logarithm

where `maxPsd` is the maximum power spectral density in dBm/MHz reported for the frequency range overlapping the channel, and `channel_bandwidth_MHz` is the channel bandwidth in MHz (20, 40, 80, 160, or 320).

3. Takes the minimum of the two EIRP values:

ch_max_power = min(AFC_EIRP, EIRP_from_PSD)

4. Enforces a minimum floor value indexed by bandwidth from the AFC_MIN_PSD table:

| Bandwidth (MHz) | Minimum EIRP Floor (dBm) |

|-----|-----|

| 20 | 14 |

| 40 | 17 |

| 80 | 20 |

| 160 | 23 |

| 320 | 26 |

5. Final assignment: **ch_max_power = max(AFC_MIN_PSD[bandwidth], int(min(psdEirp[ch], maxEirp[j])))**

Novel Contribution 4: Sub-Channel EIRP Propagation

The invention provides a sub-channel expansion algorithm that propagates wide-channel EIRP limits to the constituent 20 MHz sub-channels. For each approved wide-channel center frequency index (`channelCfi`), the system identifies all 20 MHz sub-channels using channel offset deltas (`ch_delta`) specific to each bandwidth:

| Bandwidth (MHz) | ch_delta Offsets from Center |

```

|-----|-----|
| 20 | [0] |
| 40 | [2] |
| 80 | [2, 6] |
| 160 | [2, 6, 10, 14] |
| 320 | [2, 6, 10, 14, 18, 22, 26, 30] |

```

For each delta `d`, both `channelCfi - d` and `channelCfi + d` are checked against the list of valid 20 MHz channels in the 6 GHz band and assigned the computed `ch_max_power`.

Novel Contribution 5: 320 MHz Mode 1 / Mode 2 Disambiguation

The invention addresses the specific technical challenge of 320 MHz channel plan disambiguation in the 6 GHz band. The 6 GHz spectrum supports two non-overlapping 320 MHz channel plans:

- **Mode 1:** Center channels at 31, 95, 159 (covering sub-channels approximately 1-61, 65-125, 129-189)
- **Mode 2:** Center channels at 63, 127, 191 (covering sub-channels approximately 33-93, 97-157, 161-221)

Some 20 MHz sub-channels appear in the sub-channel sets of both Mode 1 and Mode 2. The invention provides a mode disambiguation function `get_mode(channel)` that:

1. If the channel appears only in Mode 1's sub-channel range, assigns Mode 1
2. If the channel appears only in Mode 2's sub-channel range, assigns Mode 2
3. If the channel appears in both ranges (overlap region), computes the distance from the channel to the nearest Mode 1 center channel and the nearest Mode 2 center channel, and assigns the mode whose center is closer

During sub-channel expansion, sub-channels are only assigned EIRP values if their mode matches the mode of the wide-channel authorization.

Novel Contribution 6: WGS84-ECEF Geolocation Validation

The invention provides a geolocation change detection method using Earth-Centered, Earth-Fixed (ECEF) coordinate transformation. When an AP submits a new AFC request with updated GPS coordinates, the system compares the new coordinates against the coordinates used in the existing AFC proxy registration. The comparison proceeds as:

1. Convert both old and new (latitude, longitude) to ECEF (x, y, z) using WGS84 ellipsoid parameters:
 - Semi-major axis: **$a = 6,378,137.0$ meters**
 - Flattening: **$f = 1 / 298.257223563$**
 - Semi-minor axis: **$b = a * (1 - f)$**

- Normal radius of curvature: $N = a / \sqrt{1 - f(2 - f) * \sin^2(\text{lat})}$
- ECEF coordinates: $x = (N + \text{alt}) \cos(\text{lat}) \cos(\text{lon})$; $y = (N + \text{alt}) \cos(\text{lat}) \sin(\text{lon})$; $z = (N(1 - f)^2 + \text{alt}) \sin(\text{lat})$
- 2. Compute Euclidean distance in the XY plane: $\text{distance} = \sqrt{(x_{\text{new}} - x_{\text{old}})^2 + (y_{\text{new}} - y_{\text{old}})^2}$
- 3. Compute absolute height difference: $\text{height_diff} = |h_{\text{new}} - h_{\text{old}}|$
- 4. If both $\text{distance} \leq \text{threshold}$ AND $\text{height_diff} \leq \text{threshold}$, the location is considered unchanged
- 5. The system-level location threshold is **200 meters** (configurable per site policy)

When a location change is detected, the system:

1. Deletes the existing AFC proxy registration for the AP
2. Constructs a new payload with the updated GPS coordinates
3. Submits a new POST request to the AFC proxy
4. Retrieves the updated channel authorization via GET

Novel Contribution 7: AP Lifecycle Management in AFC

The invention provides automatic lifecycle management of AP registrations within the AFC system. When the RRM event processing system receives notification that an AP has transitioned to an abnormal state:

- **AP_UNCLAIMED**: The AP has been factory-reset or disassociated from its organization; the system deletes the AFC proxy registration and purges the geolocation cache entry (`devices/{apID}/gps`)
- **AP_UNASSIGNED**: The AP is not yet assigned to a site; the system deletes the AFC proxy registration and purges the geolocation cache

The deletion operation uses an HTTP DELETE request with the `X-HTTP-Method-Override: DELETE` header, with up to 3 retry attempts and a 1-second delay between retries.

Novel Contribution 8: LPI Fallback on GPS Unavailability

The invention provides a graceful degradation mechanism for AFC-capable APs that cannot obtain GPS coordinates. When the AFC channel validation function returns status "NoGPS" and the AP is configured with the `lpi_ok` capability flag:

1. The system disables Standard Power mode for this AP (`afc_standard_power = False`)
2. Clears the AFC channel authorization (`afc_channels = {}`)
3. Falls back to the AP's configured channel list (which may be used in LPI mode)
4. Sets `afc_source = "lpi_fallback"` for audit tracking
5. Records the NoGPS status in Redis for retry processing by the AFC tick process

This mechanism preserves 6 GHz connectivity for affected users rather than disabling the radio entirely.

Novel Contribution 9: Redis-Backed AFC State Management

The invention provides a Redis-based state management layer for AFC information. AFC node state is stored using structured Redis hash entries with keys encoding organization, site, and AP identifiers. The system maintains:

- AFC authorization status (DONE, NotDONE, NoGPS)
- Channel authorization with per-channel EIRP values
- Authorization expiration timestamp
- Bandwidth configuration
- Site and organization identifiers

A periodic tick process (`tick_process_rrm_afc`) iterates over all AFC nodes tracked in Redis, re-validates their geolocation, refreshes authorizations near expiry, and removes entries that have been successfully re-authorized.

Novel Contribution 10: Multi-Environment Provider Support

The invention supports multiple AFC provider backends through a flag-to-provider-name mapping:

Flag	Provider Name	Purpose
----- ----- -----		
"0"	"wfa"	Production Wi-Fi Alliance AFC system
"1"	"wfa-th01"	Internal test harness
"2"	"wfa-th02"	Certification test harness
"dev"	"dev"	Development/QA environment (uses mock responses)

When the provider flag is "dev", the system bypasses the AFC proxy entirely and uses either a mock AFC response stored in Redis or a built-in default response template, enabling full end-to-end testing without an active AFC system.

Detailed Description — Part A: AFC Core Architecture

Overview of the Preferred Embodiment

The following description sets forth specific details of the preferred embodiment of the invention. Like reference numerals refer to like elements throughout. The description is organized to follow the data flow from AFC payload construction, through proxy communication, to channel availability extraction and EIRP assignment.

The preferred embodiment is implemented in Python and deployed as a set of microservices and stream-processing components within a cloud-managed enterprise wireless platform. The AFC-specific components reside in a module package designated `src/afc/`, comprising three primary modules: `afc_payload.py`, `afc_query.py`, and `afc_utils.py`, integrated with the broader RRM system through `rrmAFC.py` and `rrmACSV2.py`.

Section 1: AFC Payload Construction (`afc_payload.py`)

1.1 Channel-to-Frequency Reference Mapping

The module defines a reference dictionary `channel_to_freq` that maps each 20 MHz channel index in the 6 GHz band to its center frequency in MHz. This mapping covers all 59 channels allocated for unlicensed use under 47 CFR Part 15, Subpart E. The complete mapping is:

Table 1: 6 GHz Band Channel-to-Center-Frequency Mapping

Channel	Center Freq (MHz)	Channel	Center Freq (MHz)	Channel	Center Freq (MHz)
-----	-----	-----	-----	-----	-----
1	5955	85	6375	169	6795
5	5975	89	6395	173	6815
9	5995	93	6415	177	6835
13	6015	97	6435	181	6855
17	6035	101	6455	185	6875
21	6055	105	6475	189	6895
25	6075	109	6495	193	6915
29	6095	113	6515	197	6935
33	6115	117	6535	201	6955
37	6135	121	6555	205	6975
41	6155	125	6575	209	6995

Channel	Center Freq (MHz)	Channel	Center Freq (MHz)	Channel	Center Freq (MHz)
45	6175	129	6595	213	7015
49	6195	133	6615	217	7035
53	6215	137	6635	221	7055
57	6235	141	6655	225	7075
61	6255	145	6675	229	7095
65	6275	149	6695	233	7115
69	6295	153	6715		
73	6315	157	6735		
77	6335	161	6755		
81	6355	165	6775		

All channels are spaced 4 channel units apart (equivalent to 20 MHz), starting from channel 1 at 5955 MHz center frequency (corresponding to the 5945-5965 MHz band edge pair). The allocation ends at channel 233 (7115 MHz center), corresponding to the 7105-7125 MHz range.

1.2 Channel Frequency Range Mapping by Bandwidth

The invention defines a comprehensive channel frequency range mapping `get_channel_freq_mapping()` that provides, for each supported bandwidth, the set of valid channel center indices and their corresponding lower and upper frequency boundaries. This mapping is essential for computing PSD-to-EIRP conversions, as described in Section 3.

Table 2: 20 MHz Channel Frequency Ranges (selected)

Channel	Low (MHz)	High (MHz)
-----	-----	-----
1	5945	5965
5	5965	5985
33	6105	6125
97	6425	6445
229	7085	7105

Table 3: 40 MHz Channel Frequency Ranges (selected)

Channel	Low (MHz)	High (MHz)
-----	-----	-----
3	5955	5995
11	5995	6035
131	6595	6635
227	7075	7115

Table 4: 80 MHz Channel Frequency Ranges (complete)

Channel	Low (MHz)	High (MHz)
-----	-----	-----
7	5975	6035
23	6035	6115
39	6115	6195
55	6195	6275
71	6275	6355
87	6355	6435
103	6435	6515
119	6515	6595
135	6595	6675
151	6675	6755
167	6755	6835
183	6835	6915
199	6915	6995
215	6995	7075

Table 5: 160 MHz Channel Frequency Ranges (complete)

Channel	Low (MHz)	High (MHz)
-----	-----	-----

Channel	Low (MHz)	High (MHz)
15	5955	6115
47	6115	6275
79	6275	6435
111	6435	6595
143	6595	6755
175	6755	6915
207	6915	7075

Table 6: 320 MHz Channel Frequency Ranges (complete)

Channel	Low (MHz)	High (MHz)	Mode
-----	-----	-----	-----
31	5945	6265	1
63	6105	6425	2
95	6265	6585	1
127	6425	6745	2
159	6585	6905	1
191	6745	7065	2

1.3 AFC Request Payload Construction

The function `get_payload_proxy(ap_id, site_id, gps, provider_flag, afc_certificationId)` constructs a standards-compliant AFC request payload. The function signature with default values is:

```
get_payload_proxy(
    ap_id="sampleDevice",
    site_id="sampleSite",
    gps={},
    provider_flag="0",
    afc_certificationId=[{"id": "2AHBN-AP64", "rulesetId":
"US_47_CFR_PART_15_SUBPART_E"}]
)
```

Payload Construction Logic:**Step 1 — Orientation Normalization:**

The ellipse orientation angle is extracted from the GPS data. If the orientation value exceeds 180 degrees, it is reduced by 180 degrees to normalize it to the range $[0^\circ, 180^\circ]$ as required by the AFC protocol specification.

```
orientation = gps.get("location", {}).get("ellipse", {}).get("orientation", 45.0)
if orientation > 180:
    orientation = orientation - 180
```

Step 2 — Location Parameter Extraction with Safety Bounds:

- Height above ground: `max(gps.get("elevation", {}).get("height", 3.0), 0.1)` — minimum 0.1 meters to prevent invalid zero-height submissions
- Vertical uncertainty: integer value, default 2 meters
- Latitude: from GPS ellipse center, default 37.3228934
- Longitude: from GPS ellipse center, default -122.0322895 (Sunnyvale, CA area)
- Major axis: `min(325, int(gps.get("ellipse", {}).get("majorAxis", 100)))` — capped at 325 meters
- Minor axis: `min(325, int(gps.get("ellipse", {}).get("minorAxis", 50)))` — capped at 325 meters

Step 3 — Payload Template Assembly:

The complete payload template includes:

```
{
  "requestId": "<AP MAC address>",
  "siteId": "<site UUID>",
  "providerName": "<resolved from provider_flag>",
  "deviceDescriptor": {
    "serialNumber": "<AP MAC address>",
    "certificationId": [
      {
        "id": "2AHBN-AP64",
        "rulesetId": "US_47_CFR_PART_15_SUBPART_E"
      }
    ]
  },
  "location": {
    "elevation": {
      "height": 3.0,
      "heightType": "AGL",
      "verticalUncertainty": 2
    },
    "ellipse": {
```

```

    "center": {
      "longitude": -122.0322895,
      "latitude": 37.3228934
    },
    "majorAxis": 100,
    "minorAxis": 50,
    "orientation": 45
  },
  "indoorDeployment": 1
},
"inquiredFrequencyRange": [
  {
    "lowFrequency": 5925,
    "highFrequency": 7115
  }
],
"inquiredChannels": [
  {"globalOperatingClass": 131},
  {"globalOperatingClass": 132},
  {"globalOperatingClass": 133},
  {"globalOperatingClass": 134},
  {"globalOperatingClass": 137}
]
}

```

The `indoorDeployment` field is set to 1, indicating indoor deployment as required for Standard Power indoor operation under 47 CFR Part 15, Subpart E.

Step 4 — Dynamic Field Population:

After assembling the template, the function populates dynamic fields:

- Sets `requestId` and `deviceDescriptor.serialNumber` to `ap_id` if provided
- Sets `siteId` to `site_id` if provided
- Resolves `providerName` from `get_flag_provider_mapping()` using `provider_flag`

1.4 Default AFC Response for Development and Testing

The function `get_default_response(ap_id, site_id)` generates a synthetic AFC response for use in development environments (when `provider_flag == "dev"`) or when no mock response is found in Redis. This response provides a predetermined set of channel authorizations that enable end-to-end testing:

Table 7: Default AFC Response Channel Authorizations

GlobalOperatingClass	Bandwidth	Channels (channelCfi)	Max EIRP (dBm)
-----	-----	-----	-----
131	20 MHz	1, 33, 137, 209	26.0, 30, 29.7, 28

GlobalOperatingClass	Bandwidth	Channels (channelCfi)	Max EIRP (dBm)
132	40 MHz	3, 75, 131, 179	29.0, 31.5, 25, 27
133	80 MHz	7, 71, 135, 199	30.2, 29.4, 26, 30
134	160 MHz	15, 143	33.2, 33.2
137	320 MHz	31, 191	33.2, 30

The response includes:

- `state` : "DONE"
- `availabilityExpireTime` : Current UTC time plus 24 hours (ISO 8601 format)
- `responseCode` : 0 (Success)
- `shortDescription` : "Success"

**

1.5 Provider Flag Mapping**

Table 8: AFC Provider Flag to Provider Name Mapping

Flag	Provider Name	Purpose
-----	-----	-----
"0"	"wfa"	Production Wi-Fi Alliance certified AFC system
"1"	"wfa-th01"	Internal test harness machine
"2"	"wfa-th02"	Certification test harness machine
"dev"	"dev"	Development environment (mock responses only)

The provider flag is stored as a site-level policy parameter (`afc-provider-flag`) and retrieved from site configuration at processing time. This allows different sites within the same organization to use different AFC backends.

**

Figure 5 — GlobalOperatingClass to Bandwidth Mapping

Before: This mapping table identifies the operating classes the controller uses to align requested bandwidth with AFC response classes. The table is important because later power and channel calculations depend on selecting the correct class before any sub-channel expansion occurs.

GlobalOperatingClass	Bandwidth MHz
---	---
131	20
132	40
133	80
134	160
137	320

After: The mapping is a compact but critical part of the implementation because every later processing stage assumes the controller has selected the correct operating class. By anchoring bandwidth handling to a known class map, the invention keeps channel parsing deterministic across providers and AP models.

Figure 6 — 6 GHz Band Channel Map (5925–7125 MHz)

Before: This figure summarizes the 20 MHz channel grid across the entire 6 GHz band used by the controller when expanding authorizations and validating overlaps. The actors here are the band itself, the channel centers, and the RRM logic that interprets returned center frequencies against this map.

Segment MHz	20 MHz channel centers
---	---
5925-6045	1, 5, 9, 13, 17, 21, 25
6045-6165	29, 33, 37, 41, 45, 49, 53
6165-6285	57, 61, 65, 69, 73, 77, 81
6285-6405	85, 89, 93, 97, 101, 105, 109
6405-6525	113, 117, 121, 125, 129, 133, 137
6525-6645	141, 145, 149, 153, 157, 161, 165
6645-6765	169, 173, 177, 181, 185, 189, 193
6765-6885	197, 201, 205, 209, 213, 217, 221
6885-7125	225, 229, 233

After: The band map connects the abstract AFC response to concrete channel objects used by the controller. It sets up the detailed EIRP and 320 MHz mode logic, both of which rely on channel-center arithmetic over this same 20 MHz lattice.

Figure 19 — Multi-Provider Support Architecture

Before: This architecture figure explains how a single policy flag changes the provider path without altering AP firmware or the high-level controller workflow. The figure is important because it captures a practical deployment benefit: production, certification, test-harness, and development backends can all be exercised through the same logic.

Flag value	Provider name	Routed backend
---	---	---
"0"	wfa	Production AFC provider
"1"	wfa-th01	Internal test harness
"2"	wfa-th02	Certification harness
"dev"	dev	Mock / default response path

After: The routing table shows how the invention separates provider choice from AP behavior. That separation supports certification workflows and development workflows while preserving the same controller-side processing pipeline and is therefore a strong differentiator for patent positioning.

Section 2: AFC Proxy Communication (afc_query.py)**

2.1 AFC Request Submission: PostAfcRequest

The function `PostAfcRequest(afc_proxy_link, sample_data)` submits a device registration request to the AFC proxy. The complete operation is:

Pre-condition Summary:

Before calling this function, the caller has constructed a valid payload using `get_payload_proxy()` with the AP's current GPS coordinates, certification ID, and provider flag.

1. If `sample_data` is None, a default payload is generated via `get_payload_proxy()`
2. An HTTP session is created with SSL certificate verification disabled (`session.verify = False`) to support internal PKI environments
3. An HTTP POST request is sent to `afc_proxy_link` with:
4. Request body: JSON-serialized `sample_data`
5. Content-Type header: `application/json`
6. Timeout: 5 seconds

7. On successful response (HTTP 200): returns `True` and logs success with `requestId` and `siteId`
8. On non-200 response: logs warning with the error status code and returns `False`
9. On network exception: logs warning "Post to AFC Proxy failed." and returns `False`

The 5-second timeout prevents hung connections from blocking the RRM processing pipeline during AFC proxy outages.

Post-condition Summary:

On success (True returned), the AP's device descriptor and location have been registered with the AFC proxy. The AFC proxy will asynchronously process this registration with the upstream AFC system. A subsequent GET request is required to retrieve the authorization result.

2.2 Channel Authorization Retrieval: `GetChannelByMac`

The function `GetChannelByMac(afc_proxy_link, mac, timeout=10)` retrieves the channel authorization for a specific AP. The URL pattern is:

```
{afc_proxy_link}/{mac}
```

For example: `http://afc-proxy-prod.mist.pvt/afc/devices/5433c60d3162`

Operation:

1. HTTP GET request to the constructed URL with 5-second timeout
2. Response is parsed as JSON
3. On exception: logs warning "Query of the AP {mac} failed, using default response for next step." and returns empty dict `{}`
4. Returns the JSON response dict on success

Pre-condition Summary:

A prior `PostAfcRequest()` must have succeeded for this MAC address. The response state field ("DONE", "NotDONE", etc.) indicates whether the AFC proxy has completed processing.

Post-condition Summary:

The returned dict contains the full AFC authorization result, including `availableChannelInfo`, `availableFrequencyInfo`, `availabilityExpireTime`, `state`, and `requestId`. The caller must check `response.get("state") == "DONE"` before using channel data.

2.3 AP Registration Deletion: `DeleteAP`

The function `DeleteAP(afc_proxy_link, mac)` removes an AP registration from the AFC proxy. This is required when an AP changes location (triggering re-registration with new GPS), or when an AP is decommissioned.

URL Pattern:

```
{afc_proxy_link}/{mac}
```

Operation with Retry Logic:

```
max_retries = 3
retry_count = 0
timeout_seconds = 5

while retry_count < max_retries:
    try:
        response = session.request(
            "DELETE",
            url=url,
            headers={"X-HTTP-Method-Override": "DELETE"},
            timeout=timeout_seconds
        )
        if response.status_code == 200:
            return True
        retry_count += 1
        if retry_count < max_retries:
            time.sleep(1)
    except Exception as e:
        retry_count += 1
        if retry_count < max_retries:
            time.sleep(1)

return False
```

The `X-HTTP-Method-Override: DELETE` header is included to support proxy servers and load balancers that may not forward HTTP DELETE verbs directly. The 1-second inter-retry delay prevents flooding the AFC proxy during transient failures. After 3 failed attempts, the function logs a warning and returns False, allowing the calling system to continue processing rather than blocking indefinitely.

Pre-condition Summary:

Called when location change is detected or AP state transitions to UNCLAIMED/UNASSIGNED. The MAC address must match an existing registration in the AFC proxy.

Post-condition Summary:

On success, the AFC proxy has removed all state associated with this MAC address. The system may subsequently submit a new POST request for the same MAC address with updated parameters.

****Section 3: Summary of AFC Core Module Interactions**

The three modules (`afc_payload.py` , `afc_query.py` , `afc_utils.py`) interact in a well-defined sequence. The following summarizes the complete flow before a diagram:

Pre-diagram Summary — AFC Core Flow:

When an RRM ACS event arrives for a Standard Power AP, the system first checks whether AFC authorization is needed. If the AP has `standard_power=True` in its radio configuration, the system calls `afc_channel_validation()` which orchestrates the following sequence:

1. **GPS Retrieval** — Queries Redis for `devices/{apID}/gps` ; if absent, returns NoGPS
2. **Payload Construction** — Calls `get_payload_proxy()` with AP ID, site ID, GPS data, provider flag, and certification ID
3. **Proxy GET** — Calls `GetChannelByMac()` to check for existing authorization
4. **Location Comparison** — If existing authorization found, calls `location_match()` to detect relocation
5. **Conditional POST** — If state is not "DONE", or location changed, calls `DeleteAP()` then `PostAfcRequest()`
6. **Second GET** — After POST, calls `GetChannelByMac()` again to retrieve fresh authorization
7. **Response Processing** — Calls `process_afc_response()` to extract per-channel EIRP values
8. **Return** — Returns (`afc_channels dict`, `expiration_timestamp`, `status_string`)

Figure 2 — AFC Request Lifecycle Flowchart

Before: This flowchart explains the end-to-end request lifecycle once an AP needs 6 GHz standard-power authorization. It identifies the order in which ACS, Redis, payload generation, proxy communication, response parsing, and channel assignment occur.



After: The lifecycle is significant because the controller does more than proxy a request: it decides when AFC is necessary, validates the returned state, and converts the response into an enforceable RRM outcome. That sequencing enables the power-computation and fallback logic shown in later figures.

Figure 3 — AFC Payload JSON Structure

Before: This figure focuses on the structure of the outbound AFC payload. It shows the fields the controller derives from AP identity, site policy, and GPS data before the proxy forwards the request to a provider.

```

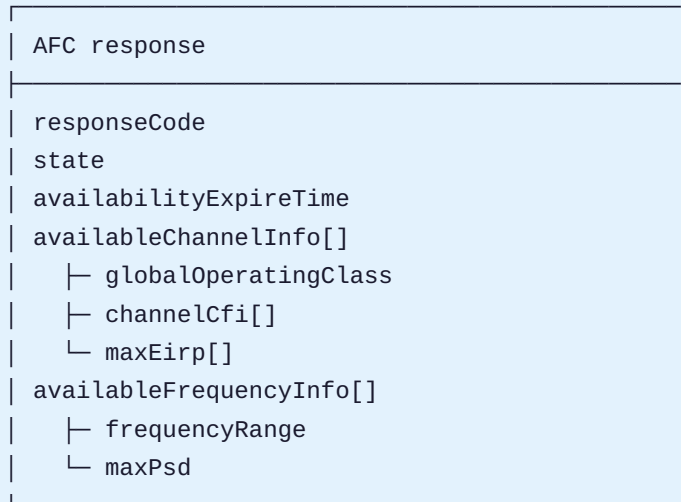
| AFC request payload |
|-----|
| requestId           : AP MAC |
| siteId              : site UUID |
| providerName        : wfa / wfa-th01 / wfa-th02 / dev |
| serialNumber        : AP MAC |
| certificationId     : 2AHBN-AP64 |
| rulesetId           : US_47_CFR_PART_15_SUBPART_E |
| majorAxis / minor   : capped at 325 m |
| height              : floor at 0.1 m |
| center latitude     : GPS latitude |
| center longitude    : GPS longitude |
| inquiredChannels    : 131,132,133,134,137 |
| inquiredFreqRange   : 5925-7115 MHz |
|-----|

```

After: The payload is novel because it is controller-synthesized and policy-driven rather than firmware-hardcoded. That gives the cloud platform leverage to switch providers, normalize geolocation parameters, and keep AP firmware unchanged while still meeting AFC protocol requirements.

Figure 4 — AFC Response Structure

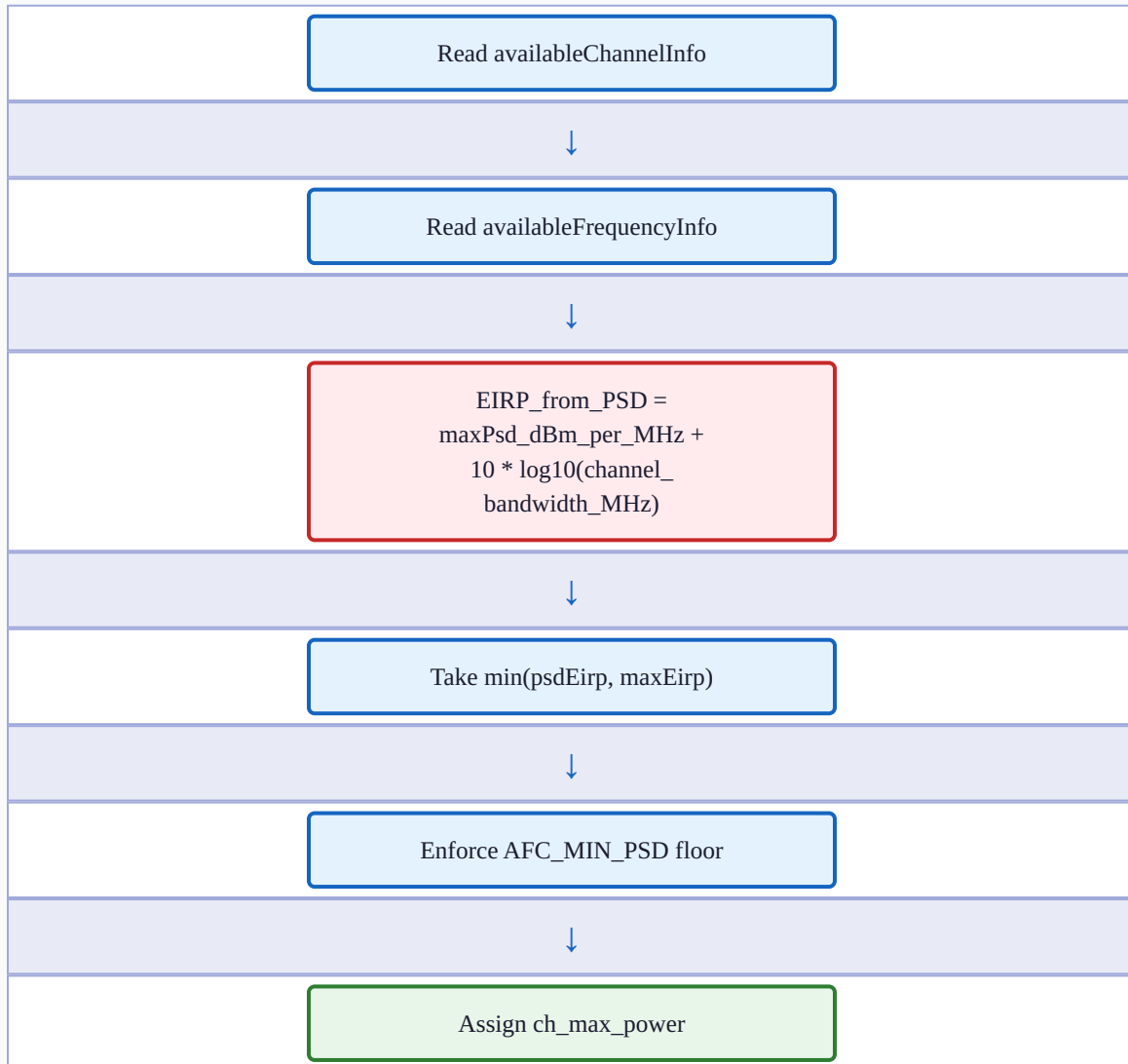
Before: This diagram shows the normalized response structure returned to the controller after proxy interaction. The controller reads both channel-level and frequency-range constraints because the implementation treats them as jointly binding.



After: The importance of the response structure lies in the simultaneous presence of maxEirp and maxPsd information. The invention exploits both fields in a controller-side computation, which distinguishes the disclosed system from simpler controller or device flows that trust only one returned value.

Figure 7 — EIRP Computation Flow

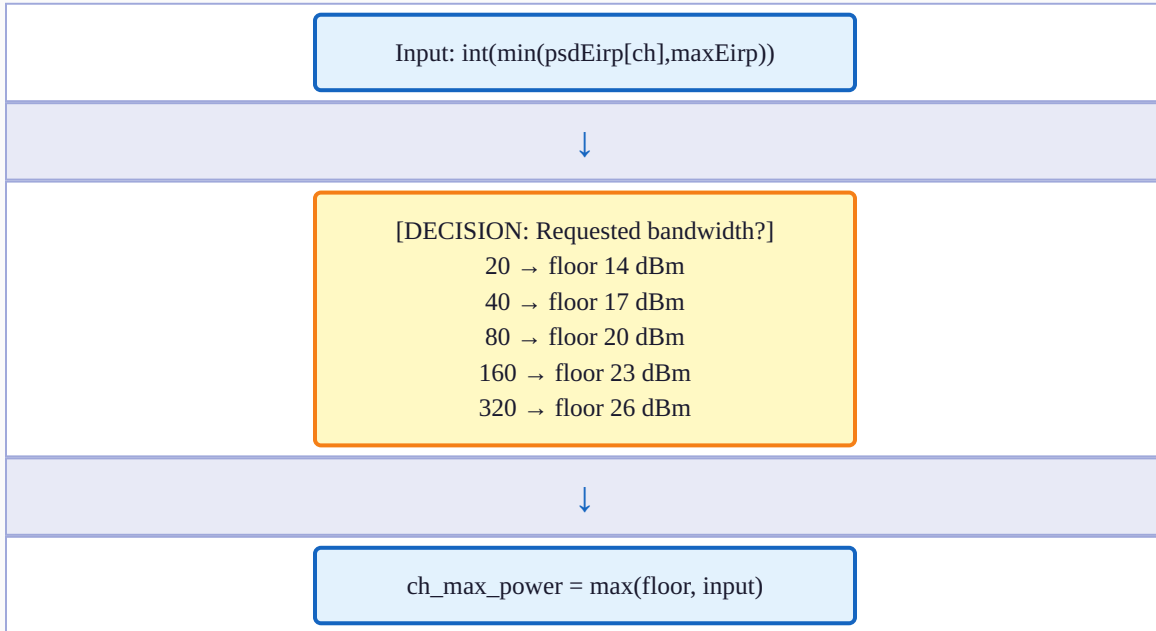
Before: This flow shows the exact order in which the controller derives the final per-channel power assignment. The sequence begins with the provider response, computes PSD-derived values, compares them against maxEIRP, and finally enforces the AFC_MIN_PSD floor.



After: The significance of the flow is that the final power value is never taken directly from a single AFC field. Instead, the controller applies layered regulatory constraints, which creates the formal foundation for the independent claims directed to power computation.

Figure 8 — AFC_MIN_PSD Floor Enforcement Decision

Before: This decision diagram isolates the floor-enforcement branch that occurs after the controller has already selected the lower of PSD-derived EIRP and AFC maxEIRP. It identifies the bandwidth-indexed constants that guarantee the final value does not fall below the implementation floor.



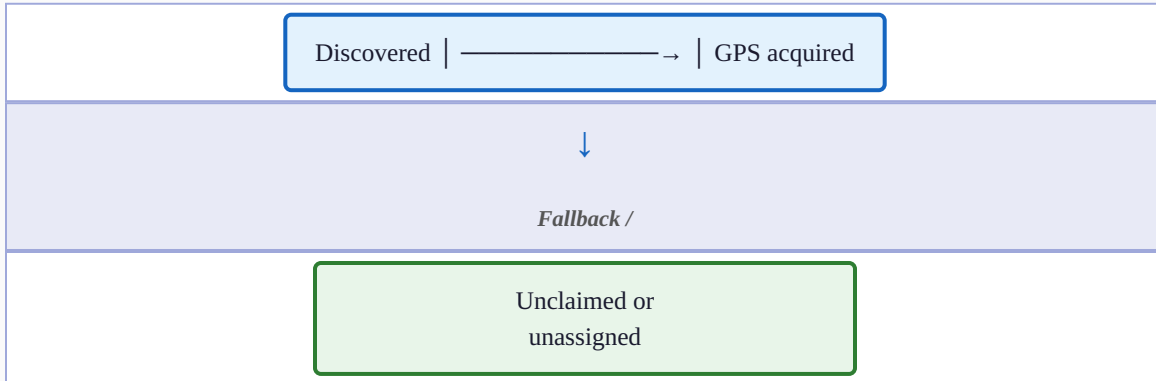
After: This figure demonstrates that floor enforcement is deterministic and bandwidth specific, not an ad hoc adjustment. It directly supports the patent distinction that the controller implements a dual-constraint plus floor-enforcement chain rather than relying on a single reported power limit.

Post-diagram Summary — Caller Responsibilities:**

After receiving the `(afc_channels, expiration_time, afc_status)` tuple, the caller in `rrmACSV2.py` intersects the AFC channels with the AP's configured channels via `channels_validation()`, updates the ACS command with AFC channel data, and emits the final RRM decision including the per-channel EIRP cap (`afc_max_eirp`) for firmware enforcement.

Figure 17 — AP Lifecycle in AFC System

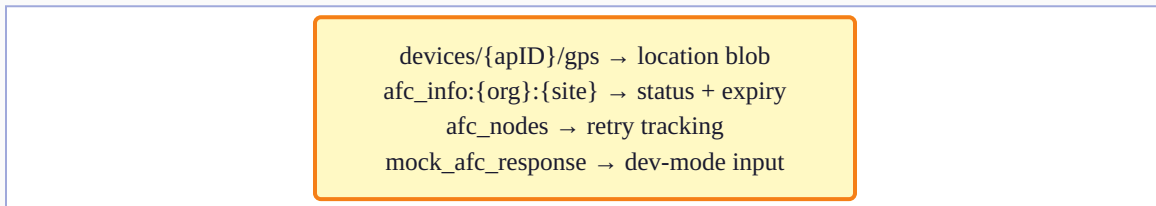
Before: This lifecycle diagram presents the major operational phases an AFC-capable AP moves through in the disclosed system. The phases tie together normal authorization, fallback behavior, relocation handling, and end-of-life cleanup.



After: The importance of the lifecycle view is that AFC is treated as a continuing operational state, not a single authorization event. That perspective explains why cleanup, retry, relocation, and fallback logic are all claimed as parts of the same inventive system.

Figure 18 — Redis Cache Architecture for AFC State

Before: This figure shows how Redis is used as the persistent working memory for the AFC subsystem. Separate key spaces hold GPS information, authorization status, expiry, and queue state so that event-driven and periodic logic can coordinate through shared data.



After: Redis is not merely a cache in this design; it is the state backbone that lets event processing and tick processing cooperate. That cloud-state design is a major difference from AP-local AFC handling and supports the fleet-management claims.

Detailed Description — Part B: RRM-AFC Integration

Section 4: RrmAFCBolt — Distributed Stream Processing Component (rrmAFC.py)

The `RrmAFCBolt` class is an Apache Storm BasicBolt component that processes AFC-related events in the distributed stream processing topology. It operates as a dedicated processing node focused exclusively on AFC coordination tasks, separating AFC lifecycle management from the main ACS channel selection pipeline.

4.1 Bolt Initialization

Pre-initialization Summary:

The `initialize(stormconf, context)` method is called once when the bolt is deployed into the Storm topology. All stateful objects are created here and maintained across tuple processing calls.

The initialization establishes:

1. **Environment Detection:** Reads `env` from Storm configuration (values: "staging", "prod", "eu"). EU environments use a longer DFS rollback time of 2700 seconds (45 minutes) vs. 2100 seconds (35 minutes) for other regions.
1. **API Rate Limits:** Sets `papi_limit = 200` (maximum AP records per API call); reduces to 20 in staging environments to prevent overloading test infrastructure.
1. **API Endpoint Configuration:** Resolves the Mist API URL from `papi.url` or `papi` configuration keys, defaulting to `papi-internal-{ENV}.mist.pvt`.

```
http://afc-proxy-{ENV}.mist.pvt/afc/devices
```

1. **AFC Proxy URL:** Reads `afc-proxy.url` from Storm configuration, defaulting to:
1. **Redis Connections:** Establishes two separate Redis connections:
 2. `redis_conn` : RRM state database (from `redis.rrm-proxy.location`)
 3. `location_redis_conn` : GPS/geolocation database (from `redis.location.location`)
1. **Request Headers:** Sets `X-FROM` header to the topology name (e.g., "rrm-main-tick") for request tracing and audit purposes.
1. **Standard Power Cache:** Initializes `ap_standard_power_cache = {}` — an in-memory dictionary that caches AP standard power configuration to avoid repeated API calls.

4.2 Event Processing: `process(tup)`

Pre-processing Summary:

The `process(tup)` method handles two types of Storm tuples: tick tuples (periodic timer events) and data tuples (AP state change notifications). Tick tuples trigger the `process_tick()` method. Data tuples contain JSON-encoded maps of AP IDs to their abnormal state information.

Data Tuple Processing Steps:

1. **Tick Check:** If `tup.is_tick_tuple()` is True, delegate to `process_tick()` and return.
1. **JSON Parsing:** Extracts and parses `tup.values[-1]` as a JSON object containing an `abnormal_ap_map` — a dictionary mapping AP MAC addresses to state information dicts or legacy state strings.
1. **Format Normalization:** Handles both old format (string state value) and new format (dict with `ap_status`, `site_id`, `org_id` fields).
1. **State Filtering:** Only processes APs with `ap_status` in `["AP_UNASSIGNED", "AP_UNCLAIMED"]`. Other abnormal states (e.g., `AP_DISCONNECTED`) are not handled by this bolt.
1. **Cache-Augmented Filtering:** For qualifying APs, `site_id` and `org_id` are retrieved from the `ap_standard_power_cache` if available (providing richer context than may be in the event), falling back to the values in the event.
1. **Event Handler Dispatch:** Calls `ap_event_handler(afc_proxy_url, location_redis_conn, redis_conn, filtered_ap_map)`.

Post-processing Summary:

After processing, the AP's AFC proxy registration has been deleted and the geolocation cache has been cleared. The AP will need to re-register when it comes back online as a claimed, assigned device.

4.3 Tick Processing: `process_tick()`

Pre-tick Summary:

The tick process runs periodically (at the Storm topology tick interval) to process APs that are tracked in Redis as needing AFC re-authorization. These are APs that previously failed AFC validation (e.g., NoGPS status) and need to be retried.

The tick calls `tick_process_rrm_afc(afc_proxy_url, location_redis_conn, redis_conn, rrm_policy_default)`.

Tick Process Steps:

1. Retrieves all AFC nodes from Redis via `rrmRedis.get_afc_info(redis_conn)`.
1. For each AP in the AFC node list:
 - a. Parses the stored AFC info JSON to extract `org_id`, `site_id`, `bandwidth`.
 - b. Attempts geolocation retrieval via `get_ap_geolocation(location_redis_conn, ap_id)`.

- c. If GPS not found: Updates Redis status to "NoGPS" and continues to next AP.
- d. Retrieves site policy to get provider flag and location threshold.
- e. **Cleans up stale registration:** Calls `DeleteAP(afc_proxy_url, ap_id)` before re-attempting.
- f. Calls `afc_channel_validation(...)` with current GPS data.
- g. If status is "DONE": Removes AP from AFC tracking in Redis (`del_afc_info`).
- h. If status is not "DONE": Updates Redis status with new status value.

Post-tick Summary:

APs that achieved "DONE" status are removed from the retry queue. APs that remain in non-DONE states are kept for the next tick cycle.

4.4 Standard Power Detection: `is_standard_power(ap_id)`

This method determines whether a given AP is configured for Standard Power operation, which requires AFC coordination.

Algorithm:

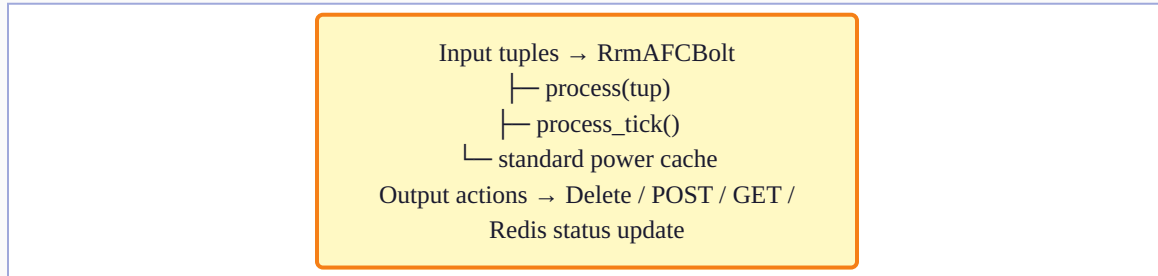
1. Check `ap_standard_power_cache` — if AP is cached, return True immediately.
1. Query Mist API for 2.4 GHz band configuration (`getRRMByMAC(ap_id, "r24", mist_api_url)`).
1. Check `radio_config.standard_power` field in the AP configuration.
1. If not found in 2.4 GHz band, query 6 GHz band configuration (`getRRMByMAC(ap_id, "r6", mist_api_url)`).
1. If `standard_power` is True: Cache the AP with timestamp, site_id, and org_id in `ap_standard_power_cache` .
1. Return the `standard_power` boolean.

The in-memory cache prevents repeated API calls for the same AP during high-frequency event processing.

**

Figure 12 — RrmAFCBolt Storm Topology

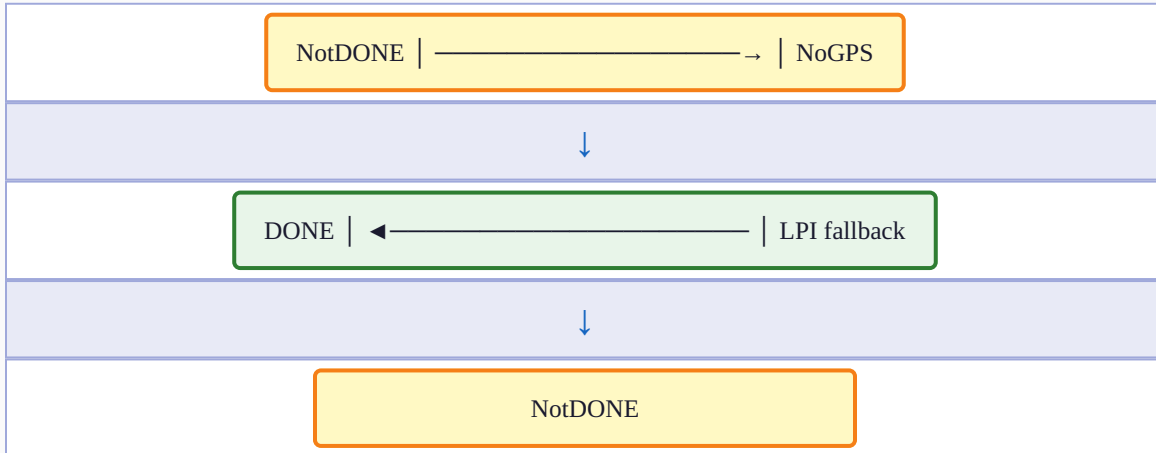
Before: This architecture diagram focuses on the Storm-processing component that turns AFC from a one-shot API call into a fleet-management service. The major actors are the event stream, the tick loop, and the Redis-backed queues that remember APs needing follow-up work.



After: The figure underscores that AFC handling is distributed and asynchronous, which is a key cloud-scale differentiator. The same bolt can react to AP events, refresh expiring entries, and coordinate cleanups without pushing those responsibilities into the AP itself.

Figure 13 — AFC Channel Validation State Machine

Before: This state machine summarizes the controller-visible outcomes of AFC validation. The states matter because subsequent ACS behavior, retry behavior, and fallback behavior all depend on which state is currently stored for the AP.



After: By separating DONE, NotDONE, NoGPS, and LPI fallback, the controller can make targeted remediation decisions instead of applying a single failure policy. That statefulness is one of the elements that makes the disclosed architecture viable at cloud scale.

Section 5: AP Abnormal State Handler (afc_utils.py — ap_event_handler)**

Pre-handler Summary:

The `ap_event_handler(afc_proxy_url, location_redis_conn, redis_conn, abnormal_ap_list)` function processes a batch of APs in abnormal states. For each AP, it determines the appropriate remediation action based on the AP's status.

Processing for AP_UNCLAIMED or AP_UNASSIGNED:

1. **AFC Proxy Deletion:** Calls `afc_query.DeleteAP(afc_proxy_url, ap_id)` with retry logic (up to 3 attempts).
1. **RRM Redis Cleanup:** Calls `rrmRedis.del_afc_info(redis_conn, ap_id)` to remove AFC tracking entry.
1. **GPS Cache Cleanup:** Calls `location_redis_conn.delete(""devices/{apID}/gps";")` to purge the geolocation entry. This is critical: if the GPS entry is retained and the same physical MAC address is reassigned to a different device at a different location, the old GPS coordinates would be returned and cause AFC validation to use the wrong location.

Post-handler Summary:

After handling, the AP has no registration in the AFC proxy and no GPS entry in the location cache. When the AP is later reclaimed or reassigned and reconnects, it will be treated as a fresh registration.

Section 6: AFC Integration in ACS Processing (rrmACSV2.py)

The `rrmACSV2.py` module implements the primary channel selection engine. When an AP is identified as Standard Power, the AFC integration intercepts the normal ACS processing flow and substitutes AFC-validated channels.

6.1 Standard Power Detection in ACS Flow**Pre-detection Summary:**

For each incoming ACS event, the system retrieves the AP configuration and checks for Standard Power mode in two locations:

```
afc_standard_power = (
    ap_config.get("radio_config", {}).get("standard_power", False)
    OR
    ap_config.get(f"band{usage}_specific", {}).get("radio_config", {}).get("standard_power",
False)
)
```

The dual-location check supports both simple radio configurations and band-specific configurations used for dual-radio or converted-radio AP models.

6.2 Bandwidth Selection for AFC

When a Standard Power AP is identified, the appropriate bandwidth for the AFC query is determined:

If `allow_auto_bw` is True (automatic bandwidth enabled):

```
bandwidth = (
    rrm_suggested_config.get(f"bandwidth_{usage}")
    OR ap_config.get(f"band{usage}_specific", {}).get("radio_config",
{}).get("bandwidth", 0)
    OR ap_config.get("radio_config", {}).get("bandwidth", 0)
)
```

If `allow_auto_bw` is False (manual bandwidth):

```
bandwidth = (
    ap_config.get(f"band{usage}_specific", {}).get("radio_config",
{}).get("bandwidth", 0)
```

```
OR ap_config.get("radio_config", {}).get("bandwidth", 0)
)
```

This priority ordering ensures that AI-suggested bandwidths (from the RRM optimization engine) are used when available, while falling back to manually configured values.

6.3 Certification ID and Ruleset Selection

Pre-certification Summary:

The certification ID used in the AFC payload varies by country. The system retrieves the country code for the site:

1. Checks Redis cache: `rrmRedis.get_country_code_per_site(redis_conn, user_site_id)`
2. If not cached: Queries Mist API via `getTotalApsAndWifiBandsBySite()` and caches result

Based on country code:

- **Canada ("CA"):** Uses `ised_id` with ruleset derived from country → Canadian ISSED certification
- **All others (including US):** Uses `fcc_id` with ruleset from `get_rulesetId_from_country_code()`

```
afc_certificationId = (
    [{"id": ised_id, "rulesetId": rulesetId}] if country_code == "CA"
    else [{"id": fcc_id, "rulesetId": rulesetId}]
)
```

Post-certification Summary:

The correct certification ID and ruleset are assembled into the AFC payload's `deviceDescriptor.certificationId` array, ensuring regulatory compliance for each country of operation.

6.4 AFC Channel Validation Call

The full AFC channel validation is invoked:

```
afc_channels, expiration_time, afc_status = afc_utils.afc_channel_validation(
    afc_proxy_link=self.afc_proxy_url,
    ap_id=ap_id,
    site_id=user_site_id,
    location_redis_conn=self.location_redis_conn,
    bandwidth=bandwidth,
    provider_flag=afc_provider_flag,
    afc_response=afc_response,
    afc_certificationId=afc_certificationId,
    location_threshold=location_threshold
)
```

Where `location_threshold` defaults to 200 meters from the site policy.

6.5 AFC Channel Intersection

After obtaining AFC-validated channels, the system intersects them with the AP's configured channel list:

```
afc_channels = afc_utils.channels_validation(afc_channels, ap_channels)
ap_channels = list(afc_channels.keys())
```

The `channels_validation()` function:

1. Normalizes AFC channel keys to strings
2. Iterates over `configured_channels` (from AP configuration)
3. Retains only channels present in both the AFC authorization and the configured channel list
4. Preserves the per-channel EIRP cap from the AFC authorization

Result: Only channels that are BOTH AFC-authorized AND administrator-configured are made available for RRM channel selection.

**

6.6 LPI Fallback on NoGPS**

Pre-fallback Summary:

When `afc_status == "NoGPS"` and `ap_config.get("lpi_ok", False)` is True, the system transitions the AP to LPI operation rather than disabling the radio:

```
if afc_status == "NoGPS" and ap_config.get("lpi_ok", False):
    afc_standard_power = False
    afc_channels = {}
    ap_channels = ap_config.get(f"band{usage}_specific", {}).get("channels", []) or \
        ap_config.get("channels", [])
    rrm_acs_command["afc_standard_power"] = False
    rrm_acs_command["afc_channels"] = {}
    rrm_acs_command["afc_source"] = "lpi_fallback"
```

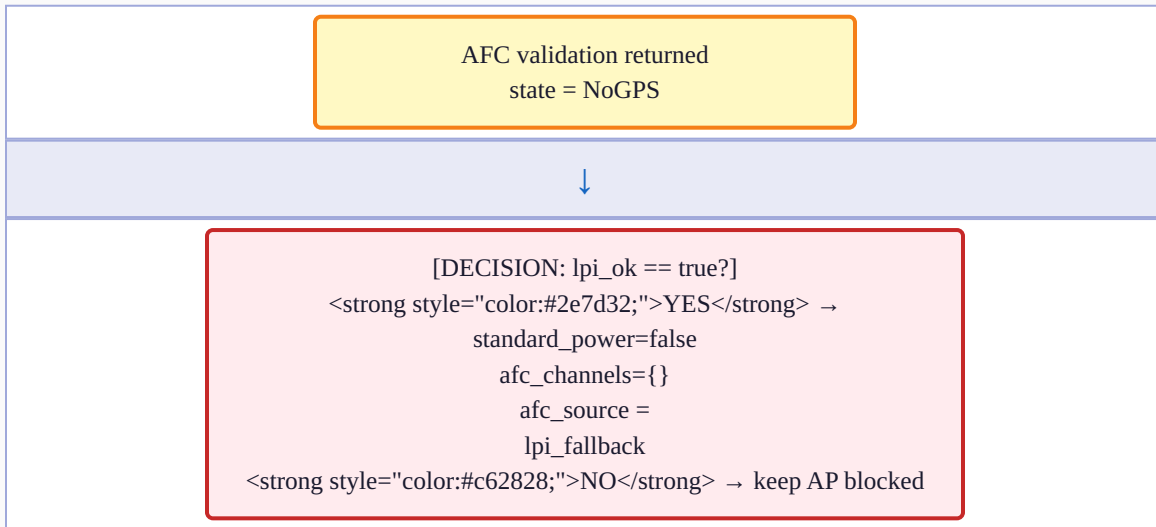
Post-fallback Summary:

The AP continues operating on its configured channels in LPI mode (lower power, no AFC required). The `afc_source = "lpi_fallback"` field provides audit trail visibility into why the AP is operating in LPI mode despite being AFC-capable hardware.

**

Figure 20 — LPI Fallback Decision Tree

Before: This decision tree concentrates on the exact condition under which the controller allows a nonstandard-power fallback instead of disabling the radio. The sequence matters because it demonstrates a service-preserving safety path that still keeps the device within compliant operating modes.



After: The fallback branch is unique because it explicitly keeps 6 GHz service available when possible rather than treating missing GPS as an all-or-nothing failure. That operational nuance is one of the clearest practical benefits of the disclosed RRM-centric design.

6.7 AFC Update Reason Handling**

When the ACS command reason is "afc-update" (periodic AFC expiry refresh):

1. Checks if AP is currently online: `ap_config.get("radio_stat", {}).get("power", 0) > 0`
2. If online and current channel is in new AFC channels: command becomes "afc-update" (EIRP limit refresh, no channel change needed)
3. If online and current channel NOT in new AFC channels: command becomes "afc-channel-update" (channel reassignment required)
4. If not online: command becomes "afc-channel-update" (prepare new channel for when AP comes online)

6.8 EIRP Enforcement in ACS Output

Pre-EIRP Summary:

After channel selection, the RRM output is augmented with AFC EIRP constraints:

```

if afc_channels:
    normalized_afc_channels = {str(k): v for k, v in afc_channels.items()}
    res["channels"] = list(normalized_afc_channels.keys())
    res["power"] = min(res["power"], normalized_afc_channels.get(str(res["channel"]),
res["power"]))
    res["rrm_power"] = res["power"]
    res["afc_standard_power"] = True
    res["afc_channels"] = afc_channels
    res["afc_expiry"] = expiration_time
    res["afc_max_eirp"] = normalized_afc_channels.get(str(res["channel"]))

```

The `res["power"]` is capped at the per-channel AFC EIRP limit. If the RRM-suggested power is below the AFC limit, the RRM power is used (the AP should not transmit at higher power than the RRM engine suggests for interference management, even if AFC would allow it). If the RRM-suggested power exceeds the AFC EIRP limit, the AFC limit takes precedence.

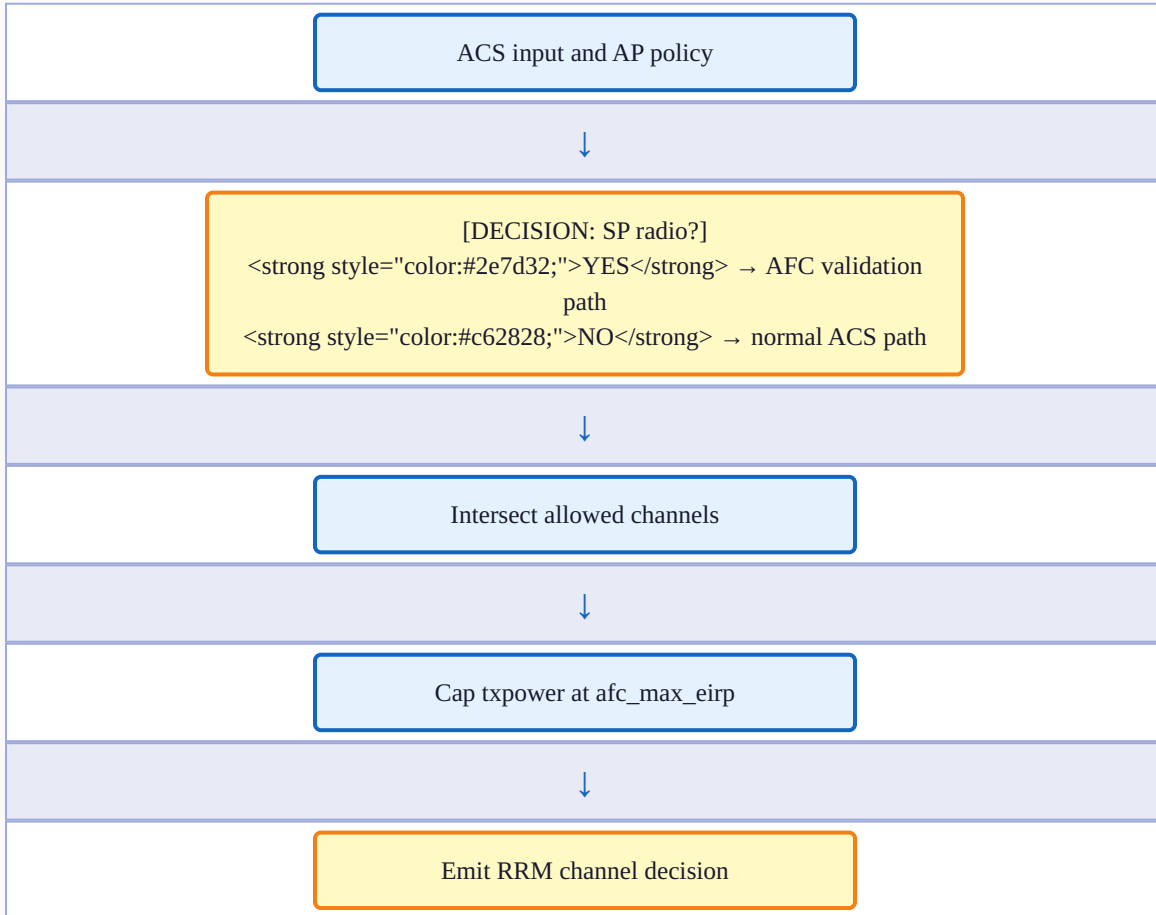
Post-EIRP Summary:

The emitted RRM result carries both the selected channel and the `afc_max_eirp` field. The AP firmware uses `afc_max_eirp` as the absolute upper bound on transmitted EIRP for the selected channel, ensuring regulatory compliance at the hardware level.

**

Figure 21 — ACS + AFC Integration Flow

Before: This figure returns to the combined ACS pipeline and shows where AFC logic is inserted into ordinary channel selection. It identifies the sequence from standard-power detection to authorization, channel intersection, and final EIRP-capped assignment.



After: The integration flow demonstrates that AFC is not a detached subsystem; it is fused into ordinary RRM decision-making. That integration is what allows the controller to convert raw AFC grants into site-aware, interference-aware, and policy-aware radio assignments.

Section 7: afcNode Processing in AP Graph (APnode.py)**

The `afcNode(g, ap_rad_id, ...)` function integrates AFC channel authorizations into the RRM topology graph. The graph `g` represents the RF topology of a site, with nodes corresponding to AP radios and edges representing RF neighbor relationships.

Pre-graph Summary:

Called during global RRM processing for each Standard Power AP radio in the site graph. The AP's AFC provider flag and FCC ID are read from the graph metadata and node attributes respectively.

Processing Steps:

```
org_id = g.graph.get("org")
site_id = g.graph.get("site")
configured_channels = g.node[ap_rad_id].get("channels", [])
rrm_bandwidth = g.node[ap_rad_id].get("rrm_bandwidth", 0)
afc_provider_flag = g.graph.get("afc-provider-flag", "0")
fcc_id = g.node[ap_rad_id].get("fcc_id") or g.graph.get("afc-fcc-id")
```

1. Node Parameter Extraction:

1. **AFC Channel Validation:** Calls `afc_utils.afc_channel_validation()` for the AP.

1. **Expiry Validation:** Checks if stored AFC authorization (from node's `afc_channels`) is still valid based on `afc_expiry` timestamp vs. current time.

```
g.node[ap_rad_id]["rrm_enabled"] = False
disable_radio(g=g, ap_rad_x=ap_rad_id, who="afc")
```

1. **No-Channel Handling:** If `afc_channels` is empty after validation:

The radio is disabled in the graph, preventing the RRM global optimization engine from assigning any channel to this AP.

```
g.node[ap_rad_id]["afc_channels"] = afc_channels
g.node[ap_rad_id]["channels"] = [int(c) for c in afc_channels.keys()]
g.node[ap_rad_id]["afc_timestamp"] = current_timestamp
g.node[ap_rad_id]["afc_expiry"] = expiration_time
g.node[ap_rad_id]["afc_source"] = "afc-proxy"
```

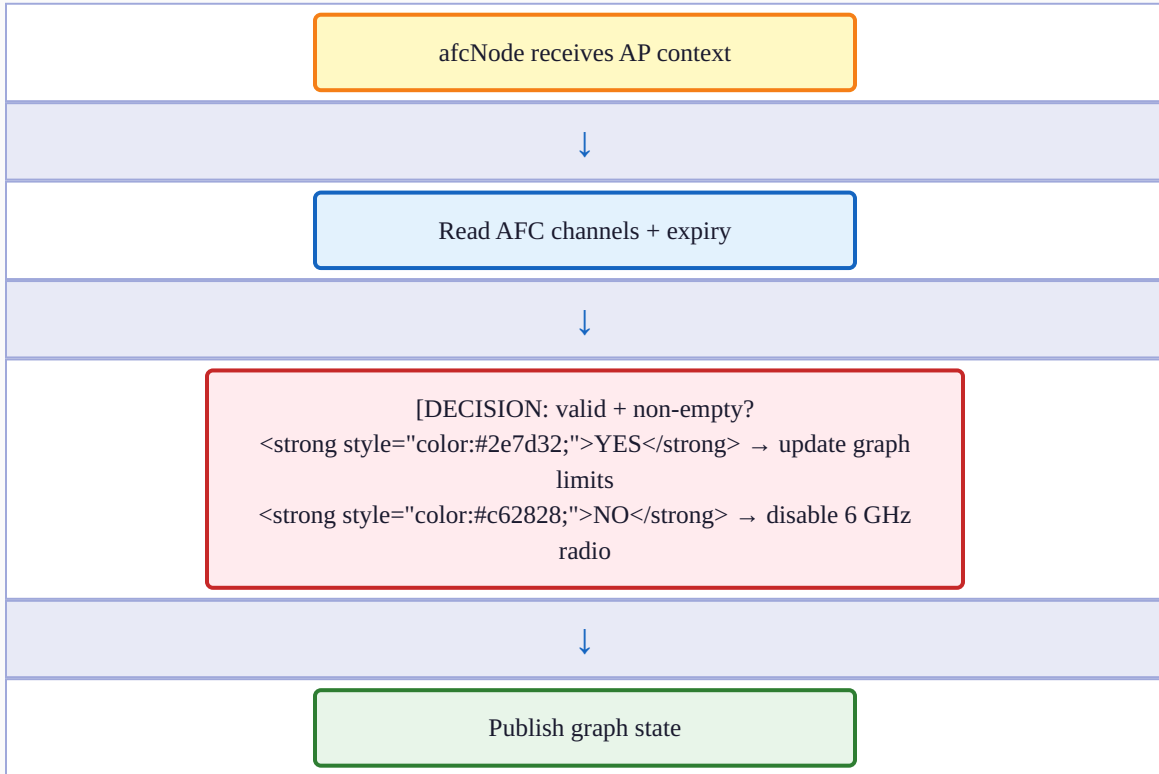
1. **Successful Authorization Handling:**

Post-graph Summary:

After `afcNode()` processing, the graph node reflects the current AFC authorization. Subsequent global RRM optimization only selects from the AFC-approved channel list for this node, ensuring all globally-optimized channel assignments are simultaneously AFC-compliant and RF-interference-optimal.

Figure 22 — afcNode Processing in APnode Graph

Before: This final figure shows how *afcNode* processing applies authorization results to the graph representation used by the wider RRM platform. The actors are the AP node, the AFC authorization record, and the graph update logic that decides whether to constrain or disable the radio.



After: The *afcNode* graph application step is the last mile of the invention: the controller turns AFC metadata into concrete graph constraints that the radio planner can enforce. This closes the loop from provider response to controller state to operational wireless behavior.

Detailed Description — Part C: Geolocation, EIRP, and Advanced Channel Logic

Before the detailed narrative below, the governing mathematical expressions are stated in plain text exactly as implemented by the disclosed controller logic:

$EIRP_from_PSD = maxPsd_dBm_per_MHz + 10 * \log_{10}(channel_bandwidth_MHz)$ (log10 is the base-10 logarithm)

$ch_max_power = \max(AFC_MIN_PSD[bandwidth], \text{int}(\min(psdEirp[ch], maxEirp[j])))$

$eccentricity_squared_e2 = f * (2 - f)$

$normal_radius_N = a / \sqrt{1 - eccentricity_squared_e2 * \sin(latitude_radians)^2}$

$x_ecef = (normal_radius_N + altitude_m) \cos(latitude_radians) \cos(longitude_radians)$

$y_ecef = (normal_radius_N + altitude_m) \cos(latitude_radians) \sin(longitude_radians)$

$z_ecef = (((1 - f)^2) normal_radius_N + altitude_m) \sin(latitude_radians)$

$xy_distance_m = \sqrt{(x_new - x_old)^2 + (y_new - y_old)^2}$

$height_difference_m = \text{abs}(height_new_m - height_old_m)$

Section 8: Geolocation Retrieval and Management

8.1 GPS Data Storage Schema

The geolocation data for each AP is stored in the location Redis database under the key:

```
devices/{apID}/gps
```

Where `{apID}` is the AP's MAC address in lowercase without delimiters (e.g., `5433c60d3162`).

The stored value is a JSON object with a top-level `location` field containing:

```
{
  "location": {
    "elevation": {
      "height": 3.0,
      "heightType": "AGL",
      "verticalUncertainty": 2
    },
    "ellipse": {
      "center": {
        "latitude": 37.3228934,
        "longitude": -122.0322895
      }
    }
  }
}
```

```

    "majorAxis": 100,
    "minorAxis": 50,
    "orientation": 45.0
  },
  "indoorDeployment": 1
}

```

8.2 Geolocation Retrieval: `get_ap_geolocation`

The function `get_ap_geolocation(location_redis_conn, ap_id)` retrieves the location data:

1. If `location_redis_conn` is `None` (connection not available), returns empty dict `{}`
2. Constructs Redis key: `"devices/{apID}/gps".format(apID=ap_id)`
3. Retrieves and JSON-parses the stored value
4. Returns `res.get("location", {})` — the location sub-object
5. On any exception (key not found, JSON parse error, Redis timeout): logs warning "No geo location for ap={ap_id}" and returns empty dict `{}`

Significance: Returning an empty dict on failure (rather than raising an exception) allows the calling code to cleanly handle the NoGPS case without try-catch at the caller level.

Section 9: Coordinate Transformation: WGS84 to ECEF

9.1 Purpose of ECEF Coordinate System

The World Geodetic System 1984 (WGS84) is the standard reference ellipsoid used for GPS coordinates. However, the spherical (latitude, longitude) representation is unsuitable for direct Euclidean distance computation because the relationship between angular differences and linear distances varies with latitude.

The Earth-Centered, Earth-Fixed (ECEF) coordinate system expresses position as (x, y, z) in meters relative to the Earth's center, with the x-axis pointing toward the prime meridian at the equator, the y-axis pointing 90° east at the equator, and the z-axis pointing toward the geographic north pole.

In ECEF coordinates, Euclidean distance in the xy-plane provides an accurate approximation of the horizontal distance between two nearby points on the Earth's surface — enabling the invention's geolocation change detection to work correctly across all latitudes.

9.2 WGS84 Ellipsoid Parameters

The WGS84 reference ellipsoid is defined by:

Parameter	Symbol	Value
-----	-----	-----
Semi-major axis	a	6,378,137.0 meters

Parameter	Symbol	Value
Flattening	f	1 / 298.257223563
Semi-minor axis	b	$a * (1 - f) = 6,356,752.314$ meters (derived)

9.3 ECEF Conversion Formulas: `lla_to_ecef`

The function `lla_to_ecef(lat, lon, alt=0)` converts geodetic coordinates to ECEF:

Step 1 — Unit Conversion:

```
lat_rad = radians(lat)
lon_rad = radians(lon)
```

Step 2 — Normal Radius of Curvature:

The normal radius of curvature N at latitude `lat_rad` is:

$$N = a / \sqrt{1 - f * (2 - f) * \sin(\text{lat_rad})^2}$$

This value represents the radius of curvature in the plane containing the surface normal and the east-west direction, varying from a at the equator to a^2/b at the poles.

Step 3 — ECEF Coordinate Computation:

```
x = (N + alt) * cos(lat_rad) * cos(lon_rad)
y = (N + alt) * cos(lat_rad) * sin(lon_rad)
z = (N * (1 - f)^2 + alt) * sin(lat_rad)
```

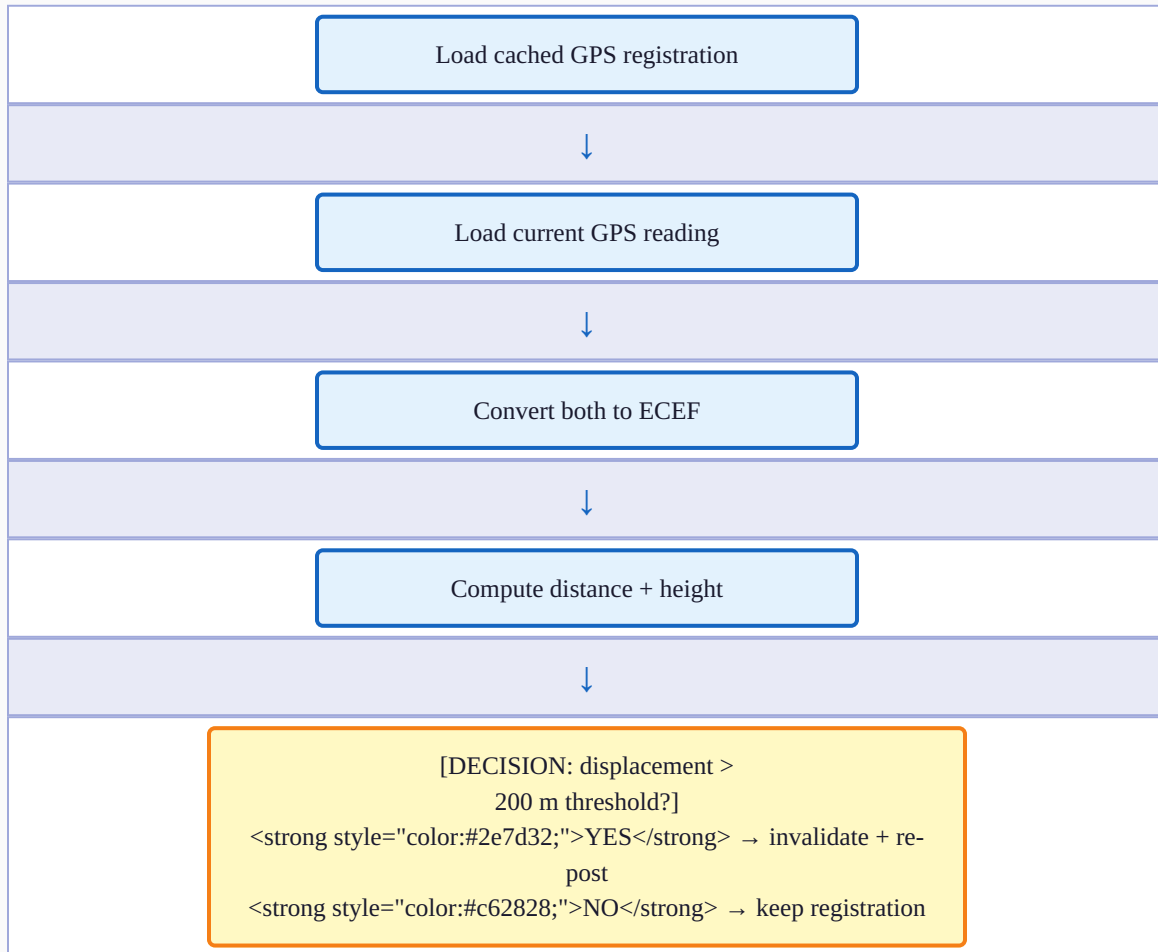
Note: The z formula uses $N * (1 - f)^2$ rather than $b^2/a^2 * N$ for computational efficiency; because $b = a * (1 - f)$, it follows that $b^2/a^2 = (a * (1 - f))^2 / a^2 = (1 - f)^2$, so the two forms are mathematically equivalent under the WGS84 definition.

Return value: Tuple `(x, y, z)` in meters.

**

Figure 9 — Geolocation Validation Flow

Before: This flowchart shows how the controller decides whether an existing AFC registration can be reused or must be replaced. The actors are GPS storage, the ECEF conversion routine, and the threshold policy that determines whether the AP has effectively moved.



After: The controller-side validation path is important because it avoids both stale authorizations and needless re-registration. It also serves as the bridge to the exact mathematical ECEF formulas shown next.

Figure 10 — WGS84 to ECEF Conversion Formula Block

Before: This formula block presents the WGS84-to-ECEF mathematics in plain text so that every variable is readable and unmasked. The formulas are used by the controller to compare current and cached geolocation data using a common earth-centered coordinate system.

```

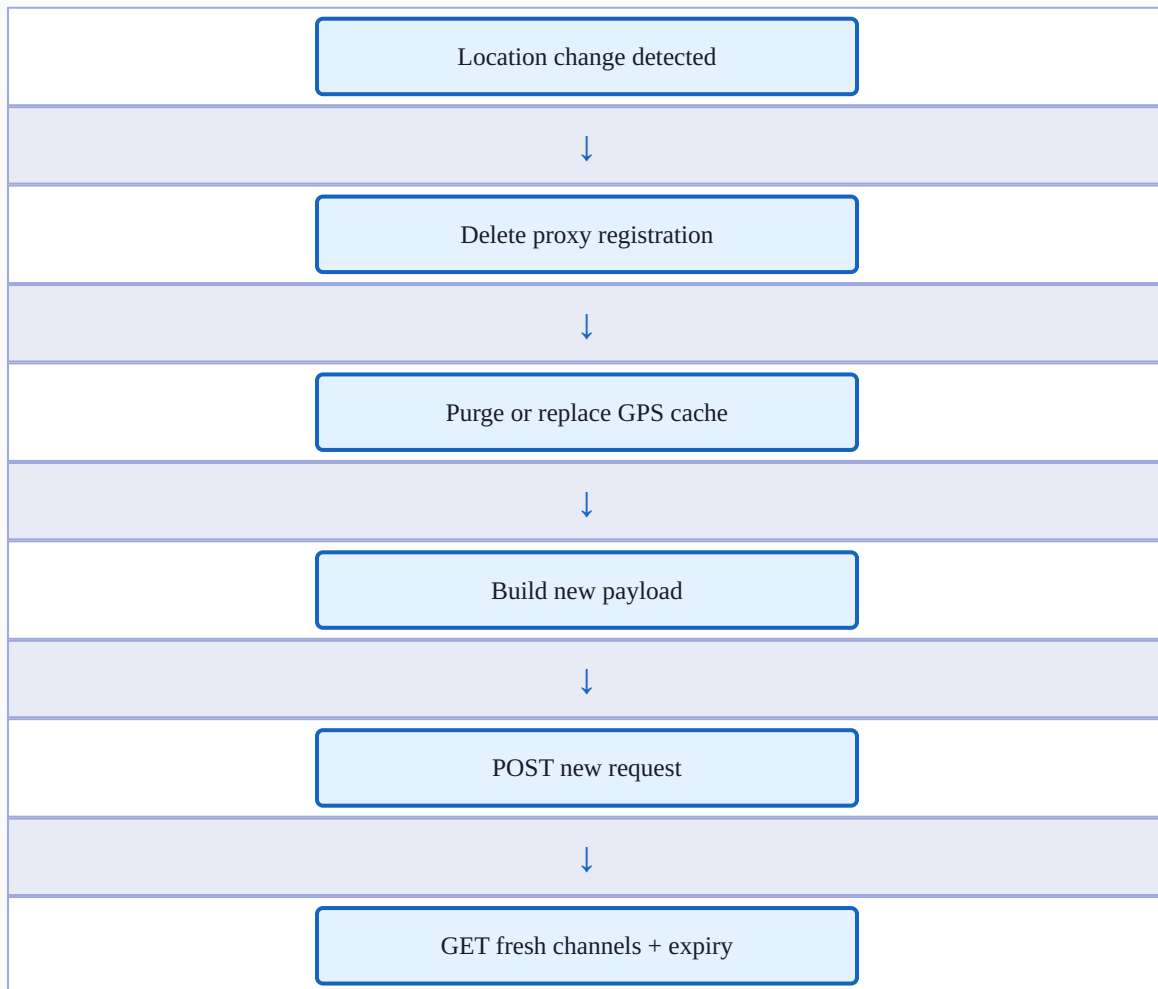
| a = 6378137.0 m |
| f = 1/298.257223563 |
| eccentricity_squared_e2 = f * (2 - f) |
| normal_radius_N = a / sqrt(1 - |
| eccentricity_squared_e2 * sin(latitude_radians)^2) |
| x_ecef = (normal_radius_N + altitude_m) * |
|         cos(latitude_radians) * cos(longitude_radians) |
| y_ecef = (normal_radius_N + altitude_m) * |
|         cos(latitude_radians) * sin(longitude_radians) |
| z_ecef = (((1 - f)^2) * normal_radius_N + altitude_m) * |
|         sin(latitude_radians) |

```

After: The mathematical block matters because the invention does not merely store location; it normalizes location into ECEF space before making lifecycle decisions. That specific mechanism is a central differentiator from prior AFC systems that treat location as static metadata.

Figure 11 — Location Change Detection and Cache Invalidation

Before: This diagram shows what happens after the controller detects a material location change. It lays out the cache invalidation and replacement sequence that keeps the AP tied to an authorization for its current position rather than a stale prior location.



After: This sequence is significant because the cache is treated as operational state, not archival state. That lifecycle connection between movement detection, registration replacement, and fresh channel retrieval is a repeated theme in the detailed description and claims.

Section 10: Location Change Detection: `location_match**`

The function `location_match(old_gps, new_gps, threshold=10)` determines whether two GPS location records represent the same physical location within a specified tolerance.

Pre-function Summary:

Called when an AP submits a new AFC request and there is an existing AFC proxy registration for the same MAC address. The old GPS is extracted from the existing proxy registration response; the new GPS is from the current Redis location entry.

Processing Steps:

1. **Null Check:** If either `old_gps` or `new_gps` is empty/None, returns False (location mismatch — re-registration required).

```
old_lat = old_gps["ellipse"]["center"]["latitude"]
old_lon = old_gps["ellipse"]["center"]["longitude"]
old_h   = old_gps["elevation"]["height"]

new_lat = new_gps["ellipse"]["center"]["latitude"]
new_lon = new_gps["ellipse"]["center"]["longitude"]
new_h   = new_gps["elevation"]["height"]
```

1. **Coordinate Extraction:**

```
old_x, old_y, _ = lla_to_ecef(old_lat, old_lon)
new_x, new_y, _ = lla_to_ecef(new_lat, new_lon)
```

1. **ECEF Conversion:**

```
height_diff = abs(old_h - new_h)
distance = sqrt((new_x - old_x)^2 + (new_y - old_y)^2)
```

1. **Distance Computation:**

```
loc_unchanged = (distance <= threshold) AND (height_diff <= threshold)
```

1. **Threshold Comparison:**

Returns `loc_unchanged` — True if location has not changed significantly.

1. **Logging:** Records distance, height_diff, threshold, and result for audit purposes (no PII in logs).

System-Level Threshold:

The function's internal default threshold is 10 meters. At the system level (in `afc_channel_validation()` and `rrmACSV2.py`), the threshold is read from site policy `location_threshold` with a default of **200 meters**. This 200-meter threshold is chosen to:

- Tolerate GPS measurement uncertainty (typical GPS accuracy for indoor deployments: 10-50 meters)
- Detect meaningful relocations (moving an AP to a different room or building) that would change the incumbent protection analysis
- Avoid false re-registrations from GPS coordinate jitter

Significance of Location Change Detection:

This capability is novel because it automatically detects and responds to AP relocations without administrator intervention. The system:

1. Calls `afc_query.DeleteAP()` to remove the stale registration
2. Constructs a new payload with the updated coordinates
3. Submits a fresh POST request
4. Retrieves the new authorization valid for the new location

This ensures that the incumbent protection analysis remains valid regardless of where an AP is physically placed.

Section 11: AFC Response Processing and EIRP Computation

11.1 Overview of `process_afc_response`

The function `process_afc_response(afc_proxy_link, res, bandwidth)` is the core algorithm that transforms an AFC system response into a per-20MHz-channel EIRP dictionary.

Pre-function Summary:

Called after a successful AFC GET response with `state == "DONE"`. The response contains two parallel data structures: `availableChannelInfo` (per-operating-class channel lists with per-channel EIRP caps) and `availableFrequencyInfo` (per-frequency-range PSD limits).

The function must reconcile these two structures — the channel-based EIRP limits and the frequency-based PSD limits — to derive the correct per-channel maximum power that satisfies both constraints simultaneously.

Processing Steps:

Step 1 — Operating Class Selection:

```
globalOperatingClass = bandwidth_to_globalOperatingClass(bandwidth)
```

This maps the requested bandwidth to the correct operating class:

Bandwidth (MHz)	Global Operating Class
-----	-----
20	131
40	132
80	133
160	134
320	137

Step 2 — Expiration Time Parsing:

```
expiration_time = res.get("availabilityExpireTime", <current UTC time>)
dt = datetime.strptime(expiration_time, "%Y-%m-%dT%H:%M:%SZ")
real_timestamp = int(time.mktime(dt.timetuple()))
```

The expiration time is converted from ISO 8601 string format to a Unix timestamp (seconds since epoch). This timestamp is stored in Redis and checked before using cached AFC authorizations.

Step 3 — Channel Class Matching:

```
for i, channelClass in enumerate(availableChannelInfo):
    if channelClass.get("globalOperatingClass") == globalOperatingClass:
```

Only the operating class matching the requested bandwidth is processed.

Step 4 — PSD-to-EIRP Computation:

```
all_channels = afc_payload.get_channel_freq_mapping()["channels"][i]["Channels"]
psdEirp = get_eirp_from_psd(all_channels, availableFrequencyInfo, bandwidth)
```

The `get_eirp_from_psd()` function computes a PSD-derived EIRP for each channel in the band, as detailed in Section 11.2.

Step 5 — Per-Channel EIRP Computation:

For each channel `ch` in `availableChannelInfo[].channelCfi[]` with corresponding `maxEirp[j]`:

```
ch_max_power = max(
    AFC_MIN_PSD.get(bandwidth, 13),
```

```
int(min(psdEirp[ch], maxEirp[j]))
)
```

The fallback value of 13 dBm is a legacy guard path that applies only if an unsupported bandwidth is supplied; the supported AFC bandwidths in this implementation are 20, 40, 80, 160, and 320 MHz.

This implements the dual-constraint with floor:

- `maxEirp[j]` : AFC system's per-channel EIRP cap (dBm)
- `psdEirp[ch]` : PSD-derived EIRP for this channel (dBm)
- `min(...)` : The more restrictive of the two EIRP limits
- `max(AFC_MIN_PSD[bandwidth], ...)` : Enforce the minimum floor so EIRP is never set below the regulatory minimum for this bandwidth

The AFC_MIN_PSD floor values are:

```
AFC_MIN_PSD = {20: 14, 40: 17, 80: 20, 160: 23, 320: 26}
```

Step 6 — Sub-Channel Expansion:

The `ch_delta` values are selected based on bandwidth:

Bandwidth (MHz)	ch_delta	Sub-channels addressed
-----	-----	-----
20	[0]	Center only ($ch \pm 0 = ch$)
40	[2]	$ch - 2, ch + 2$
80	[2, 6]	$ch - 2, ch + 2, ch - 6, ch + 6$
160	[2, 6, 10, 14]	8 sub-channels total
320	[2, 6, 10, 14, 18, 22, 26, 30]	16 sub-channels total

For each delta `d` in `ch_delta`:

```
if ch - d in channels_6G_20:
    afc_channels[ch - d] = ch_max_power
if ch + d in channels_6G_20:
    afc_channels[ch + d] = ch_max_power
```

For 320 MHz bandwidth, an additional mode filter is applied:

```

if bandwidth == 320 and wifi320Mode != get_mode(ch - d):
    continue # Skip sub-channels of the wrong mode

```

Post-function Summary:

Returns `(afc_channels, expiration_time)` where `afc_channels` is a dictionary mapping 20 MHz channel numbers (int) to maximum EIRP values (int, in dBm). This dictionary represents the complete set of AFC-approved operating parameters for the AP at its current location.

11.2 PSD-to-EIRP Conversion: `get_eirp_from_psd`

The function `get_eirp_from_psd(channel_freqs, psd_data, band_width)` computes the PSD-derived EIRP for each channel in the band.

Pre-function Summary:

The AFC response's `availableFrequencyInfo` contains a list of PSD limit entries, each specifying a frequency range and a maximum PSD value in dBm/MHz. These PSD limits constrain transmit power density across the spectrum to protect incumbents that may be affected by out-of-band emissions.

The conversion formula converts a PSD limit to an EIRP limit for a channel of given bandwidth:

EIRP_from_PSD = maxPsd_dBm_per_MHz + 10 * log10(channel_bandwidth_MHz), where `log10` denotes the base-10 logarithm

This formula follows directly from the definition of EIRP and PSD: if the spectral density is maxPsd dBm/MHz and the channel occupies channel_bandwidth_MHz MHz, then the total power (EIRP) is the product in linear scale, which becomes a sum in dB scale: 10 * log10(bandwidth) dB added to the PSD value.

For example:

- 20 MHz channel, maxPsd = 23 dBm/MHz:

$$\text{EIRP} = 23 + 10 * \log_{10}(20) = 23 + 13.01 = 36.01 \text{ dBm}$$

- 80 MHz channel, maxPsd = 20 dBm/MHz:

$$\text{EIRP} = 20 + 10 * \log_{10}(80) = 20 + 19.03 = 39.03 \text{ dBm}$$

- 160 MHz channel, maxPsd = 17 dBm/MHz:

$$\text{EIRP} = 17 + 10 * \log_{10}(160) = 17 + 22.04 = 39.04 \text{ dBm}$$

Algorithm:

1. Initialize `psd_eirp = {}` — output dictionary, channel → EIRP (dBm)
1. For each channel `(ch, ch_low, ch_high)` in `channel_freqs` :
2. Initialize `psd_eirp[ch] = 36` (default: no PSD constraint → use 36 dBm maximum)

3. Initialize `channel_eirp = 36`, `flag = False`

```
channel_eirp = min(channel_eirp, psd_value + 10 * log10(band_width))
flag = True
```

1. For each PSD entry (`psd_low`, `psd_high`, `psd_value`) in `psd_data`:
2. Check overlap: The PSD range overlaps with the channel if any of:
 3. `psd_low >= ch_low` AND `psd_low < ch_high` (PSD range starts within channel)
 4. `psd_high <= ch_high` AND `psd_high > ch_low` (PSD range ends within channel)
 5. `psd_high >= ch_high` AND `psd_low <= ch_low` (PSD range spans entire channel)
6. If overlap:
7. Early exit: If `psd_low > ch_high`, no further PSD entries can overlap this channel (PSD data is ordered by frequency)

```
psd_eirp[ch] = channel_eirp
```

1. If `flag` is True (at least one overlapping PSD entry was found):

Otherwise, the default 36 dBm remains.

Post-function Summary:

Returns `psd_eirp` dictionary mapping channel numbers to PSD-derived EIRP limits. Channels with no overlapping PSD constraints retain the 36 dBm default (equivalent to no PSD constraint from the AFC system's perspective).

**

Figure 14 — 320 MHz Mode 1 vs Mode 2 Channel Layout

Before: This figure visualizes the 320 MHz ambiguity that arises because two legal 320 MHz plans overlap at the 20 MHz sub-channel level. The controller resolves the ambiguity by comparing sub-channel proximity to the Mode 1 and Mode 2 center channels.

Mode	Center channels	Representative covered sub-channels
---	---	---
Mode 1	31, 95, 159	1-61, 65-125, 129-189
Mode 2	63, 127, 191	33-93, 97-157, 161-221

After: The figure shows why a simple wide-channel expansion is insufficient for 320 MHz authorizations. The invention adds a controller-side disambiguation step so that only sub-channels belonging to the correct mode inherit the returned power limit.

Figure 15 — ch_delta Sub-Channel Expansion per Bandwidth

Before: This table documents the exact `ch_delta` arrays that the implementation uses to expand a returned wide-channel authorization into 20 MHz child channels. The table matters because the same expansion logic feeds both channel validation and power propagation.

Bandwidth MHz	ch_delta
---	---
20	[0]
40	[2]
80	[2,6]
160	[2,6,10,14]
320	[2,6,10,14,18,22,26,30]

After: The expansion arrays are a mechanical implementation detail, but they are central to how the controller converts an AFC response into radio-usable 20 MHz entries. They also make the claims concrete by tying each wide-channel bandwidth to a deterministic sub-channel propagation rule.

Section 12: 320 MHz Mode Disambiguation**

12.1 The Mode Ambiguity Problem

The 6 GHz 320 MHz channel plan has two distinct, non-overlapping arrangements:

Mode 1 Center Channels: 31, 95, 159

- Mode 1 occupies sub-channels: approximately 1 through 61 (center 31), 65 through 125 (center 95), 129 through 189 (center 159)

Mode 2 Center Channels: 63, 127, 191

- Mode 2 occupies sub-channels: approximately 33 through 93 (center 63), 97 through 157 (center 127), 161 through 221 (center 191)

The sub-channel ranges of Mode 1 and Mode 2 overlap significantly. Channels in the overlap region (approximately 33-61, 65-93, 97-125, 129-157, 161-189) belong to both a Mode 1 and a Mode 2 320 MHz channel. When assigning sub-channel EIRP values from a Mode 1 authorization, only Mode 1 sub-channels should receive the assignment; Mode 2 sub-channels should not receive EIRP values derived from a Mode 1 AFC authorization.

12.2 Mode Range Computation

The valid sub-channel ranges for each mode are computed programmatically:

```
central_channel_mode1 = [31, 95, 159]
central_channel_mode2 = [63, 127, 191]

channels_mode1 = range(
    central_channel_mode1[0] - 30, # = 1
    central_channel_mode1[-1] + 30 + 4, # = 193
    4 # step = 4 (20 MHz spacing)
)

channels_mode2 = range(
    central_channel_mode2[0] - 30, # = 33
    central_channel_mode2[-1] + 30 + 4, # = 225
    4
)
```

12.3 Mode Assignment Algorithm: get_mode(channel)

The function `get_mode(channel)` determines the correct mode for a given 20 MHz sub-channel:

Case 1: Channel appears in `channels_mode1` only → Return 1

Case 2: Channel appears in `channels_mode2` only → Return 2

Case 3: Channel appears in BOTH `channels_mode1` and `channels_mode2` (overlap region):

```
channel_ind1 = (channel - 1) // 64
```

1. Compute the index within the Mode 1 center channel array:

```
channel_ind2 = (channel - 33) // 64
```

1. Compute the index within the Mode 2 center channel array:

```
diff_to_central1 = abs(channel - central_channel_mode1[channel_ind1])
diff_to_central2 = abs(channel - central_channel_mode2[channel_ind2])
```

1. Compute distance to nearest center:

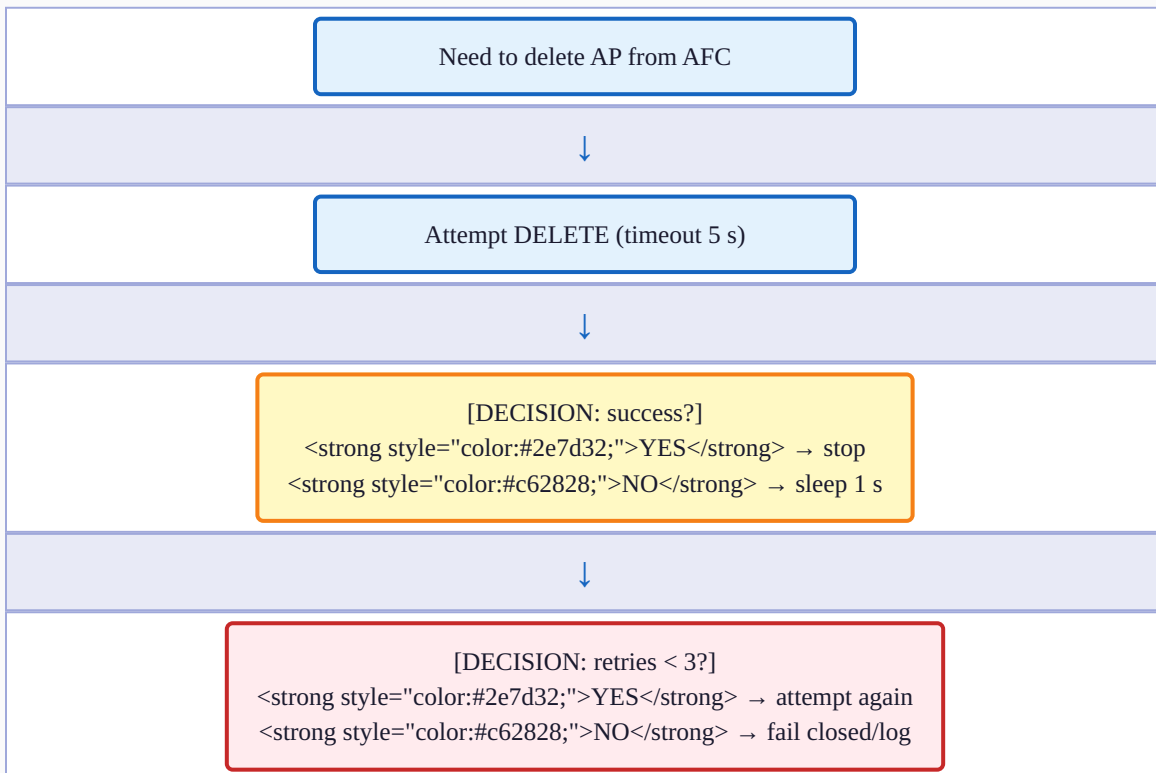
```
return 1 if diff_to_central1 < diff_to_central2 else 2
```

1. Assign to the mode whose center is closer:

This comparison ensures that an overlapping 20 MHz sub-channel is associated with the 320 MHz layout whose center channel is nearest in absolute distance.

Figure 16 — DeleteAP Retry Logic Flowchart

Before: This flowchart shows the DeleteAP cleanup sequence used when an AP becomes unclaimed, unassigned, or otherwise stale. The actors are the lifecycle event source, the DELETE client, and the retry controller that guarantees bounded cleanup behavior.



After: The retry flow is important because it turns deletion into a reliable part of the AFC lifecycle instead of a best-effort cleanup. That bounded retry behavior also distinguishes the controller-driven lifecycle from prior art that does not explicitly manage stale registrations.

Implementation Reference Table

Novel Contribution	Source File	Function / Class
-----	-----	-----
AFC proxy intermediary architecture	src/afc/afc_query.py	PostAfcRequest(), GetChannelByMac(), DeleteAP()
Structured AFC payload construction	src/afc/afc_payload.py	get_afc_req_payload(), get_flag_provider_mapping()
Dual-constraint EIRP computation	src/afc/afc_utils.py	process_afc_response(), get_eirp_from_psd()
Sub-channel EIRP propagation	src/afc/afc_utils.py	process_afc_response()
320 MHz Mode 1 / Mode 2 disambiguation	src/afc/afc_utils.py	get_mode()
WGS84-ECEF geolocation validation	src/afc/afc_utils.py	lla_to_ecef(), location_match(), get_ap_geolocation()
AP lifecycle management in AFC	src/afc/afc_utils.py	ap_event_handler()
LPI fallback on GPS unavailability	src/rrmACSV2.py	rrm_afc_channels()
Redis-backed AFC state management	src/rrmAFC.py	RrmAFCBolt, process_tick(), tick_process_rrm_afc()
Multi-provider support routing	src/rrmHandler.py and src/afc/afc_payload.py	rrm_afc_channels(), get_flag_provider_mapping()

Claims

Claim 1. A system comprising: a cloud-based radio resource management controller configured to receive automatic channel selection events for access points operating in a 6 GHz band; an Automated Frequency Coordination proxy interface configured to submit and retrieve authorization data for said access points; a payload-construction module configured to assemble an AFC request including a device identifier, a certification identifier, a ruleset identifier, WGS84 geolocation data, and inquired operating classes; and a response-processing module configured to derive per-channel operating limits for the access points from AFC response data.

Claim 2. The system of claim 1, wherein: the payload-construction module sets a default FCC identifier of 2AHBN-AP64 and a ruleset identifier of US_47_CFR_PART_15_SUBPART_E for United States standard-power operation.

Claim 3. The system of claim 1, wherein: the payload-construction module caps a semi-major axis and a semi-minor axis at 325 meters and enforces a minimum height of 0.1 meters before submitting the AFC request.

Claim 4. The system of claim 1, wherein: the AFC proxy interface addresses an endpoint matching `http://afc-proxy-{ENV}.mist.pvt/afc/devices` and uses a request timeout of 5 seconds for POST operations.

Claim 5. The system of claim 1, wherein: provider routing is controlled by a site policy flag such that `"0"` selects `"wfa"`, `"1"` selects `"wfa-th01"`, `"2"` selects `"wfa-th02"`, and `"dev"` selects `"dev"`.

Claim 6. A method comprising: receiving an AFC authorization response containing channel-level maximum EIRP data and frequency-range PSD data; identifying a requested bandwidth using GlobalOperatingClass values 131, 132, 133, 134, and 137 for 20 MHz, 40 MHz, 80 MHz, 160 MHz, and 320 MHz respectively; computing $\text{EIRP_from_PSD} = \text{maxPsd_dBm_per_MHz} + 10 * \log_{10}(\text{channel_bandwidth_MHz})$ for overlapping frequency ranges; and deriving a channel power assignment from both AFC maxEIRP and PSD-derived EIRP values.

Claim 7. The method of claim 6, wherein: the method enforces $\text{ch_max_power} = \max(\text{AFC_MIN_PSD}[\text{bandwidth}], \text{int}(\min(\text{psdEirp}[\text{ch}], \text{maxEirp}[\text{j}])))$ for each authorized channel.

Claim 8. The method of claim 6, wherein:

`AFC_MIN_PSD = {20: 14, 40: 17, 80: 20, 160: 23, 320: 26} dBm.`

Claim 9. The method of claim 6, wherein: constituent 20 MHz sub-channels are expanded by `ch_delta` arrays of `[0]`, `[2]`, `[2, 6]`, `[2, 6, 10, 14]`, and `[2, 6, 10, 14, 18, 22, 26, 30]` for 20 MHz, 40 MHz, 80 MHz, 160 MHz, and 320 MHz respectively.

Claim 10. The method of claim 6, wherein: 320 MHz assignments are filtered by a disambiguation process that selects Mode 1 center channels `[31, 95, 159]` or Mode 2 center channels `[63, 127, 191]` before applying sub-channel power propagation.

Claim 11. A method comprising: retrieving a stored registered location for an access point; converting an updated access-point location and the stored registered location from WGS84 geodetic coordinates into Earth-Centered, Earth-Fixed coordinates; computing a displacement between the locations; and determining whether AFC re-registration is required from the computed displacement.

Claim 12. The method of claim 11, wherein: the conversion uses `a = 6378137.0 m` and `f = 1/298.257223563`.

Claim 13. The method of claim 11, wherein: the method computes `normal_radius_N = a / sqrt(1 - (f (2 - f)) sin(latitude_radians)^2)`, `x_ecef = (normal_radius_N + altitude_m) cos(latitude_radians) cos(longitude_radians)`, `y_ecef = (normal_radius_N + altitude_m) cos(latitude_radians) sin(longitude_radians)`, and `z_ecef = (((1 - f)^2) normal_radius_N + altitude_m) sin(latitude_radians)`.

Claim 14. The method of claim 11, wherein: a location change is declared when horizontal displacement or height difference exceeds a site-policy threshold of 200 meters.

Claim 15. The method of claim 11, wherein: declaring the location change causes deletion of a cached AFC registration followed by a new POST request and a new GET retrieval.

Claim 16. A system comprising: an access-point lifecycle manager configured to monitor AP claim state, site assignment state, and GPS availability for a fleet of access points; a deletion client configured to remove stale AFC

registrations; and a fallback controller configured to preserve compliant 6 GHz operation when standard-power authorization cannot be maintained.

Claim 17. The system of claim 16, wherein: the deletion client issues a DeleteAP workflow with `max_retries = 3`, `timeout = 5 seconds`, and `sleep = 1 second` between retries.

Claim 18. The system of claim 16, wherein: the lifecycle manager deletes a Redis GPS key matching `devices/{apID}/gps` when an access point enters an unclaimed or unassigned state.

Claim 19. The system of claim 16, wherein: when AFC validation returns a NoGPS status and an `lpi_ok` flag is true, the fallback controller disables standard-power mode, clears AFC channel state, and falls back to Low Power Indoor operation.

Claim 20. A non-transitory computer-readable medium storing instructions that, when executed by one or more processors, cause the one or more processors to: receive AFC-triggering events from a cloud-managed wireless network; route AFC requests to one of multiple providers according to site policy; compute channel power limits from both per-channel EIRP and PSD-derived EIRP constraints; re-register an access point when ECEF-based displacement indicates movement beyond a threshold; and delete stale AFC registrations during access-point lifecycle transitions.

Abstract of the Disclosure

The disclosure describes a cloud-managed Automated Frequency Coordination architecture for enterprise 6 GHz wireless networks in which Radio Resource Management logic, rather than access-point firmware alone, controls the AFC lifecycle. A controller retrieves AP geolocation data, builds standards-compliant AFC requests, routes those requests through a configurable proxy, computes per-channel power limits from both maxEIRP and PSD-derived constraints, and applies the resulting limits to channel-selection decisions. The controller further detects AP movement with WGS84-to-ECEF conversion, automatically re-registers moved devices, cleans up stale registrations during AP lifecycle transitions, supports provider-flag routing, and downgrades to Low Power Indoor operation when GPS is unavailable but `lpi_ok` permits continued compliant service.